Performance Tuning Workshop

Mehr Speed mit den richtigen Algorithmen und Datenstrukturen

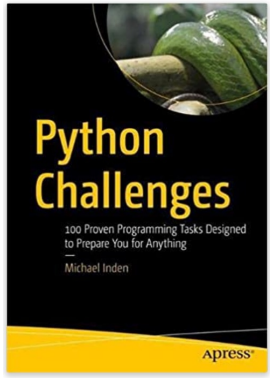
WJAX München 2026 – Michael Inden

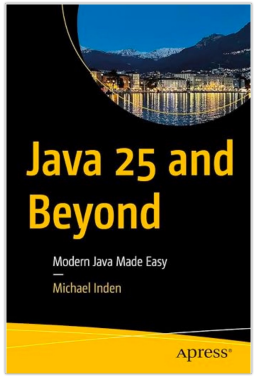
https://github.com/Michaeli71/WJAX-Performance-Tuning-Workshop-2026

Michael Inden

Speaker Intro Michael Inden

- Michael Inden, Jahrgang 1971, michael_inden@hotmail.com
- Diplom-Informatiker, C.v.O. Uni Oldenburg
- ~8 ¼ Jahre SSE bei Heidelberger Druckmaschinen AG in Kiel
- ~6 ¾ Jahre TPL, SA bei IVU Traffic Technologies AG in Aachen
- ~4 ¼ Jahre LSA / Trainer bei Zühlke Engineering AG in Zürich
- ~3 Jahre TL / CTO bei Direct Mail Informatics / ASMIQ in Zürich
- ~3 ½ Jahre Head of Development bei Adcubum in Zürich
- Ab 2026 Dozent Software Engineering und Studienleiter MAS SE
- Freiberuflicher Trainer und Konferenz-Speaker
- Autor bei dpunkt.verlag, O'Reilly und APress

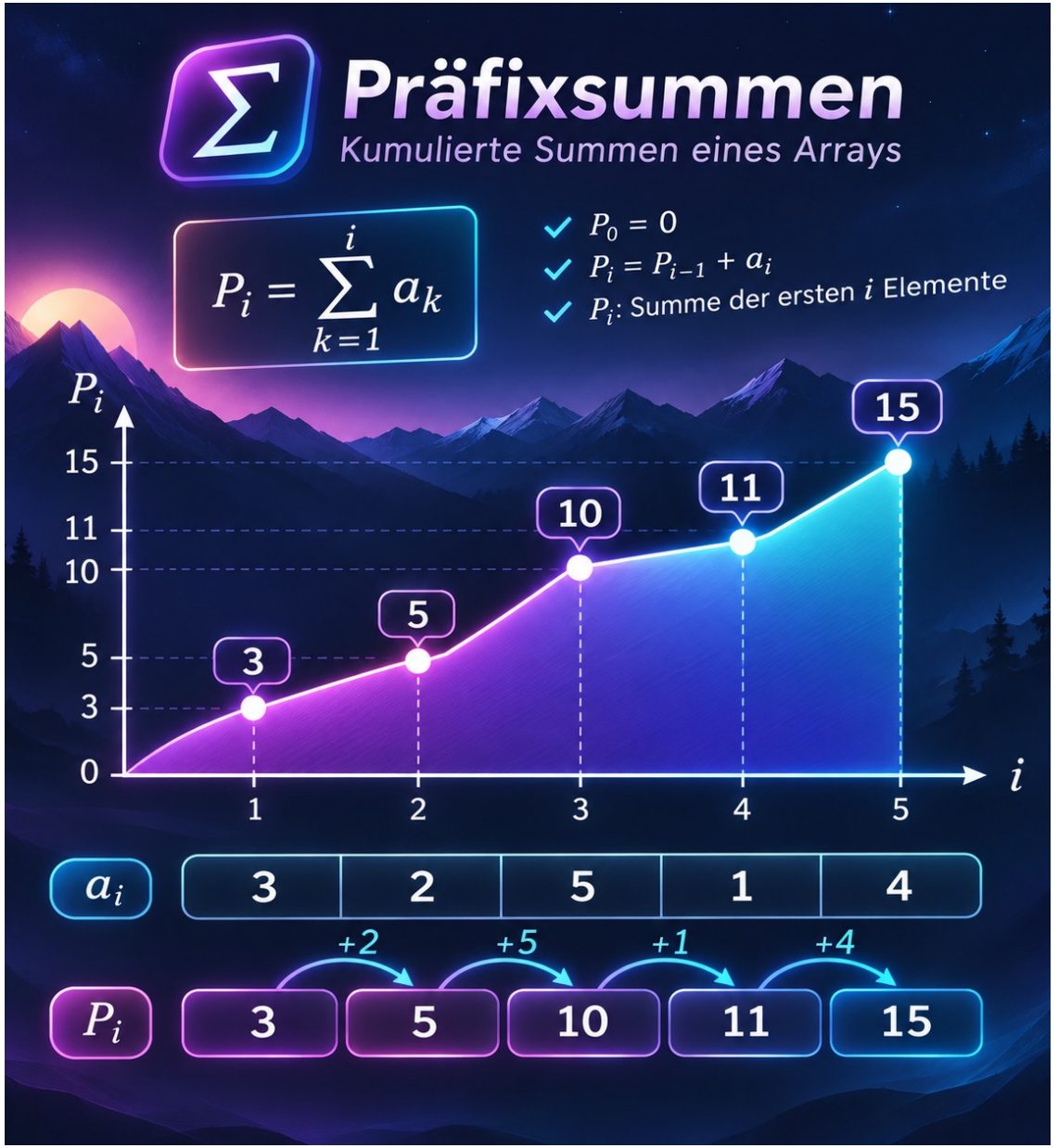




Agenda heute

1. 🎬 Einstieg
2. 🧠 Performance verstehen: Big O und Skalierung
3. 🗃️ Wichtige Datenstrukturen und ihre Eigenschaften
4. 🔎 Suchen und Sortieren
5. 💡 Cleverere Lösungen: Two Pointers und HashMaps
6. **Σ Präfixsummen**
7. 🪟 **Sliding Windows**
8. 🧩 **Rekursion und Memoization**
9. 🌳 **Tries und Autocomplete**
10. 🏁 **Performance-Finale: messen, erklären, optimieren**
11. 🔥 **Performance Fireworks**
12. 🎯 **Fazit**

Σ Präfixsummen



Bereichssummen

- Gegeben sei ein Datenbestand für den wir in einem Indexbereich `start` und `end` (inklusive) die Summe der dort enthaltenen Werte berechnen wollen.
- Brute-Force-Implementierung:

```
static long sumInRrange(List<Integer> values, int start, int endInclusive) {
    long total = 0;
    for (int i = start; i <= endInclusive; i++) {
        total += values.get(i);
    }
    return total;
}
```

- Bestimmt die Summe der Elemente von Index `start` bis Index `end`: O(n)
- Für jede weitere Anfrage muss man die Berechnung vollständig neu ausführen, also O(n x m)

Präfixsummen

- **Präfixsummen** werden aus einer Reihe von **Eingabewerten berechnet**.
- Jedes **Element** enthält die **Summe** aller **vorherigen Werte** einschließlich des aktuellen Werts.
- Dabei gelten allgemein diese Formeln:

```
result[0] = values[0]
result[1] = values[0] + values[1]
result[2] = values[0] + values[1] + values[2]
result[3] = values[0] + values[1] + values[2] + values[3]
...
```

- Durch diese Vorberechnung lassen sich viele Berechnungen für Bereichssummen deutlich effizienter durchführen, weil man nicht jedes Mal die Werte erneut aufsummieren muss.

Präfixsummen berechnen

- Die Implementierung geschieht straight forward wie folgt:

```
static List<Integer> prefixSum(List<Integer> values) {
    List<Integer> result = new ArrayList<>();
    int sum = 0;
    for (int i = 0; i < values.size(); i++) {
        sum += values.get(i);
        result.add(sum);
    }
    return result;
}


List<Integer> values = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9);
IO.println(prefixSum(values));
```

```
[1, 3, 6, 10, 15, 21, 28, 36, 45]
```

Bereichssummen basierend auf Präfixsummen berechnen

- Gedankliches Modell: Nimm die Summe bis `start - 1` und bis `end`:

```
0 -------------- start - 1
0 ------------------------------------------- end
```

- Wird die kleinere Präfix-Summe von der größeren abgezogen, bleibt genau die Summe des gewünschten Bereichs übrig:

```
                                start -------------------- end
```

- Die Implementierung ist trivial, wenn man Präfixsummen vorberechnet hat:

```
static long sumInRangeInclusive(List<Integer> prefixSums, int start, int endInclusive) {
    if (start == 0)
        return prefixSums.get(endInclusive);

    return prefixSums.get(endInclusive) - prefixSums.get(start - 1);
}
```

Präfixsummen mit dem Stream-API berechnen

```
public static long[] prefixSums(int[] values) {
    return Arrays.stream(values)
                 .asLongStream()
                 .boxed()
                 .gather(Gatherers.scan(() -> 0L, Long::sum))
                 .mapToLong(Long::longValue)
                 .toArray();
}

public static List<Integer> prefixSums(List<Integer> values) {
    return values.stream()
                 .gather(Gatherers.scan(() -> 0, Integer::sum))
                 .toList();
}
```

Was ist schneller?

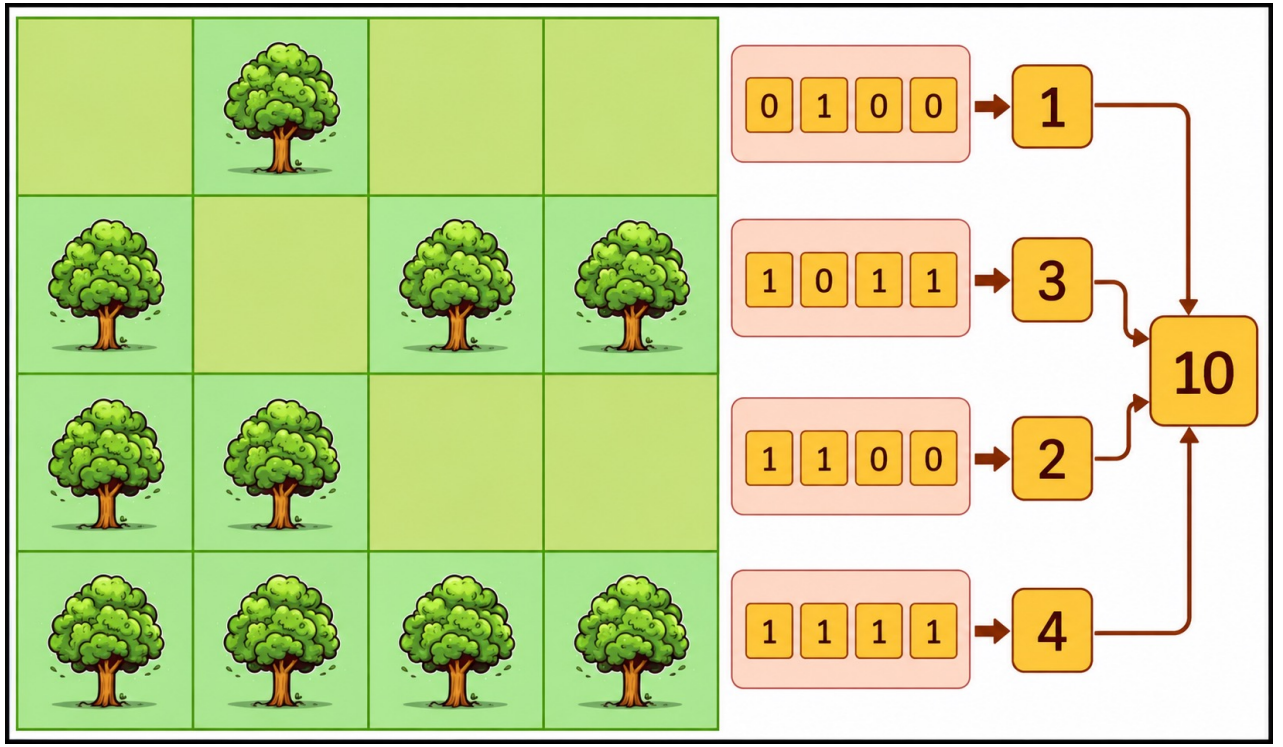
Geschwindigkeit for-Schleife vs Stream-API

```
--- 100000 Elemente ---
Stream + Gatherer          0.821 ms
For-Schleife               0.475 ms
For-Schleife ist 1.73x schneller

--- 1000000 Elemente ---
Stream + Gatherer          8.336 ms
For-Schleife               4.438 ms
For-Schleife ist 1.88x schneller

--- 10000000 Elemente ---
Stream + Gatherer        136.950 ms
For-Schleife              55.948 ms
For-Schleife ist 2.45x schneller
```

Präfixsummen in 2D

Bereichs- und Präfixsummen in 2D

- Die Grundidee funktioniert auch für zweidimensionale Daten, etwa zur Berechnung
  - **Fassungsvermögen** eines **Stausees** basierend auf **Tiefenangaben** aufgeteilt pro **m²**
  - **Baumbestand** in einem **Waldstück**, mit **Bäumen** pro **Flächeneinheit**, etwa 10 m × 10 m (**100 m²**)



Bereichs- und Präfixsummen in 2D

- Ausgangsdaten als 2D-Matrix wie diese:

0	0	2	1	2
0	1	3	5	6
1	4	2	7	0
2	5	4	0	2
0	1	0	1	1

- Bereichssummen:

0	0	2	1	2
0	**1**	**3**	**5**	6
1	**4**	**2**	**7**	0
2	**5**	**4**	**0**	2
0	1	0	1	1

1 + 3 + 5 + 4 + 2 + 7 + 5 + 4 + 0
= 31

0	**0**	2	1	2
0	**1**	3	5	6
1	**4**	2	7	0
2	**5**	4	0	2
0	**1**	0	1	1

0 + 0 + 0 + 1 + 1 + 4 + 2 + 5 + 0 + 1
= 14

0	0	2	1	2
0	1	3	**5**	**6**
1	4	2	**7**	**0**
2	5	4	**0**	**2**
0	1	0	1	1

5 + 6 + 7 + 0 + 0 + 2
= 20

Bereichs- und Präfixsummen in 2D

0	0	2	1	2
0	**1**	**3**	**5**	6
1	**4**	**2**	**7**	0
2	**5**	**4**	**0**	2
0	1	0	1	1

$1 + 3 + 5 + 4 + 2 + 7 + 5 + 4 + 0$

= 31

0	**0**	2	1	2
0	**1**	3	5	6
1	**4**	2	7	0
2	**5**	4	0	2
0	**1**	0	1	1

$0 + 0 + 0 + 1 + 1 + 4 + 2 + 5 + 0 + 1$

= 14

0	0	2	1	2
0	1	3	**5**	**6**
1	4	2	**7**	**0**
2	5	4	**0**	**2**
0	1	0	1	1

$5 + 6 + 7 + 0 + 0 + 2$

= 20

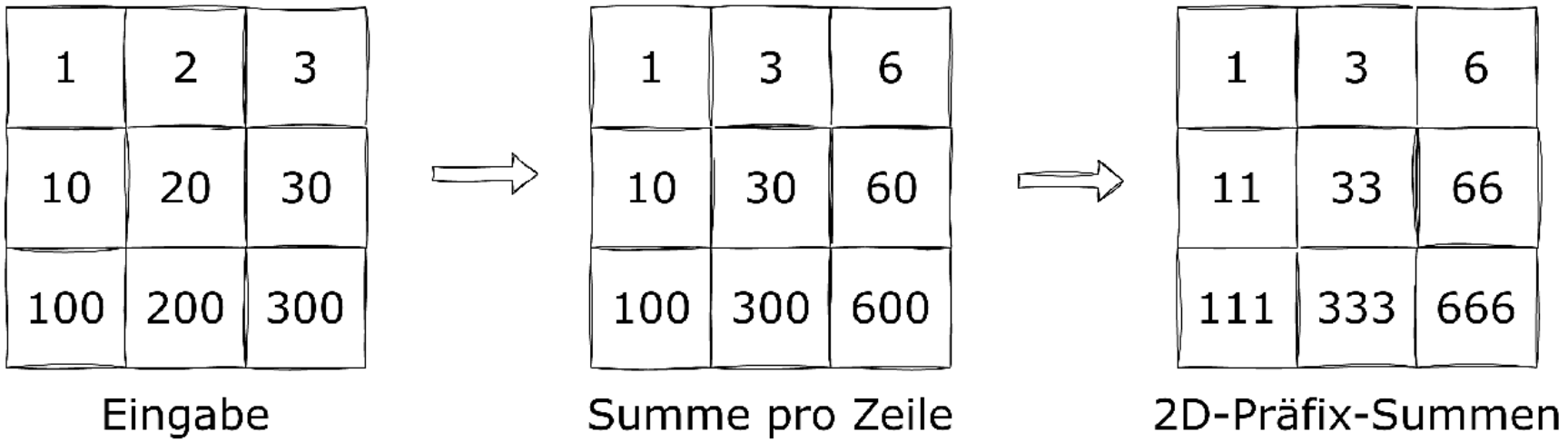
- Bereichssummen lassen sich recht einfach durch zwei verschachtelte Schleifen berechnen.

```
int result = 0;
for (int y = y1; y <= y2; y++) {
    for (int x = x1; x <= x2; x++) {
        result += values[y][x];
    }
}
```

Wenn es doch einfach ist, warum und wann sollten wir das so nicht tun?

Vorüberlegungen für Präfixsummen in 2D

- Prinzip der **eindimensionalen Präfixsummen** auf **zweidimensionale Daten** (Matrizen) übertragen.
- Für jede Zelle werden die Werte zunächst zeilenweise und anschließend kumuliert aufsummiert.
- Ergebnis: Jede Position enthält die Summe aller Elemente im Rechteck von der linken oberen Ecke bis zu dieser Zelle.



Präfixsummen in 2D

- Für jede Zelle werden die Werte zunächst zeilenweise und anschließend kumuliert aufsummiert.

```
public static int[][] prefixSum2D(int[][] values) {
    int height = values.length;
    int width = values[0].length;

    int[][] result = new int[height][width];

    for (int x = 0; x < width; x++) {
        result[0][x] = values[0][x];
        for (int y = 1; y < height; y++)
            result[y][x] = result[y - 1][x] + values[y][x];
    }

    for (int y = 0; y < height; y++) {
        for (int x = 1; x < width; x++)
            result[y][x] = result[y][x - 1] + result[y][x];
    }
    return result;
}
```

Präfixsummen in 2D abfragen

Matrix m

	0	1	2	3	4	5	6	7
0	1	2	-1	-3	4	2	0	1
1	3	-2	4	5	-3	-1	1	0
2	0	1	-3	1	3	2	4	-2
3	-1	3	4	7	-2	0	1	3
4	3	4	2	4	-7	-1	3	0
5	-5	4	1	-2	3	-4	3	3

Präfix-Summenmatrix p

	0	1	2	3	4	5	6	7
0	1	3	2	-1	3	5	5	6
1	4	4	7	9	10	11	12	13
2	4	5	5	8	12	15	20	19
3	3	7	11	21	23	26	32	34
4	6	14	20	34	29	31	40	42
5	1	13	20	32	30	28	40	45

$$\text{sum}(2,\ 2,\ 4,\ 6) = p[4][6] - p[1][6] - p[4][1] + p[1][1]$$

Berechnet die Summe im Rechteck mit obere linke Ecke $(2, 2)$ und untere rechte Ecke $(4, 6)$ (inklusive).

$$\text{sum}(ur, lc, br, rc) = p[br][rc] - p[ur-1][rc] - p[br][lc-1] + p[ur-1][lc-1]$$

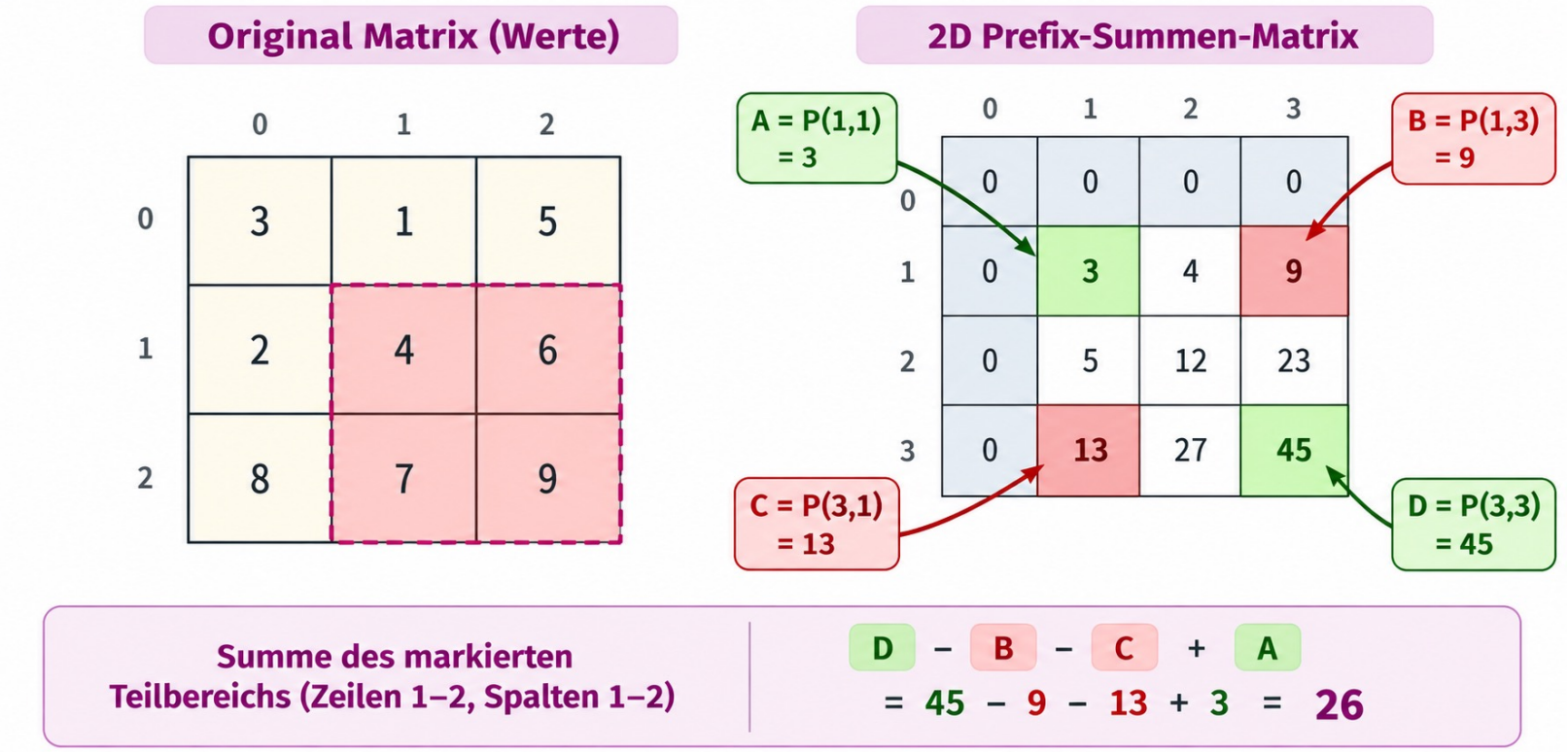
Allgemeine Formel für ein Rechteck mit obere linke Ecke (ur, lc) und untere rechte Ecke (br, rc) (inklusive).

Präfixsummen in 2D abfragen

```
public static int sum2dPrefix(int[][] prefix, int x1, int y1, int x2, int y2) {
    int result = prefix[y2][x2];

    if (x1 > 0) {
        result -= prefix[y2][x1 - 1];
    }
    if (y1 > 0) {
        result -= prefix[y1 - 1][x2];
    }
    if (x1 > 0 && y1 > 0) {
        result += prefix[y1 - 1][x1 - 1];
    }
    return result;
}
```

- **Es gibt eine optimierte Variante der Berechnung, die Null-Spalten und Null-Zeilen einfügt. Damit wird die Berechnung der Rechtecksummen vereinfacht und Sonderfälle vermieden.**

Präfixsummen in 2D leichter abfragen

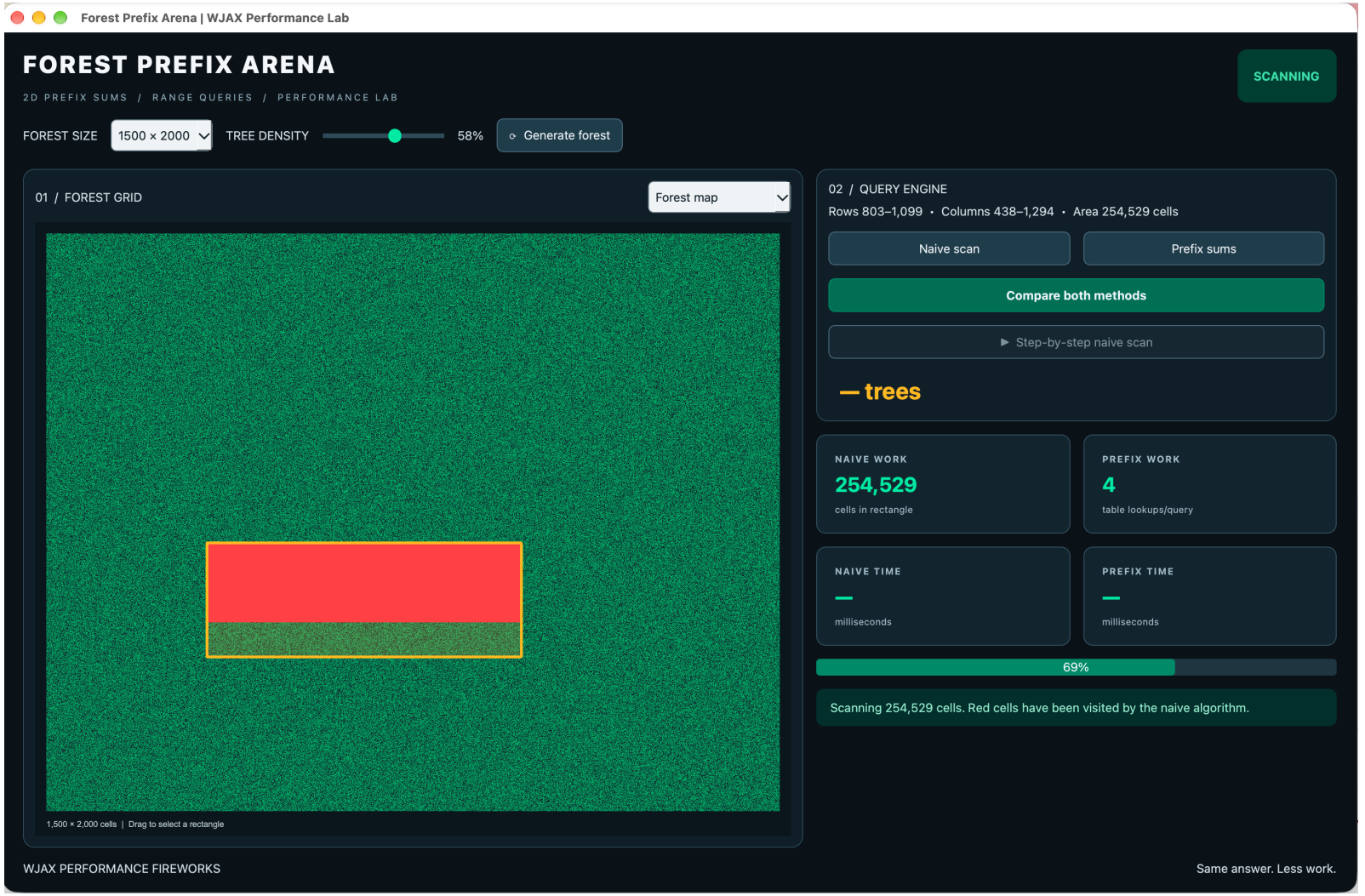



```
int sum2dUsingPrefix(int[][] prefix, int x1, int y1, int x2, int y2) {
    return prefix[y2 + 1][x2 + 1] - prefix[y1][x2 + 1] - prefix[y2 + 1][x1] + prefix[y1][x1];
}
```

DEMO: `PrefixSums2d.java`

Σ Präfixsummen

DEMO: `gui.py`



Exercises

Sliding Windows

Sliding Windows

- Wir betrachten Messwerte, zum Beispiel Besucherzahlen pro Stunde, und suchen die höchste Summe über jeweils vier aufeinanderfolgende Stunden.
- **Naiver Ansatz**: Für jeden Bereich (Wertefenster) alle darin enthaltenen Werte erneut aufsummieren.

Stunde:	0	1	2	3	4	5	6	7		
Besucher:	12	18	25	20	17	22	19	24		
Fenster 0–3:	12	18	25	20	17	22	19	24	→	12 + 18 + 25 + 20 = 75
Fenster 1–4:	12	18	25	20	17	22	19	24	→	18 + 25 + 20 + 17 = 80
Fenster 2–5:	12	18	25	20	17	22	19	24	→	25 + 20 + 17 + 22 = **84**
...										

Sliding Windows

- **Naiver Ansatz**: Für jede Fensterposition alle Werte erneut aufsummieren.
- **Und hier ist sie wieder … die Brute-Force-Implememngtierung mit zwei Schleifen ☺**

```
static int maxSumNaive(int[] values, int k) {
    if (k <= 0 || k > values.length)
         throw new IllegalArgumentException("Invalid window size");

    int max = Integer.MIN_VALUE;
    for (int i = 0; i <= values.length - k; i++) {
        int sum = 0;
        for (int j = i; j < i + k; j++) {
            sum += values[j];
        }
        max = Math.max(max, sum);
    }
    return max;
}
```

Wie könnte man ständige Neuberechnungen vermeiden?

Sliding Windows

- **Zentrale Idee**: Statt für jeden Bereich die Summe komplett neu zu berechnen, arbeiten wit mit einer Art Lupe auf den Daten und aktualisieren die Summe beim Verschieben des Fensters!

- Dazu nutzen wir die berechnete Summe wie folgt:

  1. **Alten Wert entfernen**
     Der Wert am linken Fensterrand verlässt das Fenster und wird von der bisherigen Summe abgezogen.
  2. **Neuen Wert hinzufügen**
     Wert rechts vom Fenster kommt hinzu.
  3. **Summe aktualisieren**
     Die neue Fenstersumme ergibt sich wie folgt:

$$\text{Summe}_{\text{neu}} = \text{Summe}_{\text{alt}} - \text{Wert}_{\text{alt}} + \text{Wert}_{\text{neu}}$$

Sliding Windows

- **Sliding Window**: Die Summe einmal initial berechnen und anschließend nur den herausfallenden und den neu hinzukommenden Wert berücksichtigen.

```
static int maxSumSlidingWindow(int[] values, int k) {
    if (k <= 0 || k > values.length)
         throw new IllegalArgumentException("Invalid window size");

    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += values[i];
    }
    int max = sum;
    for (int i = k; i < values.length; i++) {
        sum += values[i] - values[i - k];
        max = Math.max(max, sum);
    }
    return max;
}
```

DEMO: `MaxSumSlidingWindowExample.java`

Sliding Windows – praxisrelevante Vorteile

- **Wenn wir dieselbe Arbeit immer wieder machen …**
- **Sliding Window vor allem für riesige Datenbestände interessant, nicht für kleine k**
- **Beispiel**: 10 Millionen Messwerte, Fenstergröße 100.000

Naiv	Sliding Window
10^{12}	10^{7}
Wertbeiträge in der Größenordnung $n \cdot k$	Wertbeiträge in der Größenordnung $n + k$

- **Kernaussage**: Das Fenster verschiebt sich nur um eine Position. Deshalb müssen wir nicht jedes Mal alle Werte neu summieren.

Sliding Windows – Varianten

- **Fixed Sliding Window:** Das Fenster wird verschoben, bleibt aber immer gleich gross.

- **Variable Sliding Window:** Das Fenster wächst und schrumpft abhängig von einer Bedingung..

Sliding Windows – Kürzeste Teilsequenz mit Summe ≥ Val

- **Aufgabe:** Finde die kürzeste zusammenhängende Teilsequenz, deren Summe mindestens 7 beträgt.
- **Beispiel:** values = [2, 1, 3, 2, 4, 1, 1, 5], $S = 7$.

Sliding-Window-Update

Fenster erweitern: $sum = sum + values[r]$

Fenster verkleinern: $sum = sum - values[l]$

1 **Rechts erweitern,** bis die Bedingung erfüllt ist.

2	1	3	2	4	1	1	5

Summe = 8
Länge = 4

Summe ≥ 7: erfüllt

2 **Von links verkleinern** und bei Bedarf rechts erweitern.

2	1	3	2	4	1	1	5

Summe = 6

Zu klein – rechts erweitern

2	1	3	2	4	1	1	5

Summe = 10

2	1	3	2	4	1	1	5

Summe = 9
Länge = 3

3 **Ergebnis:**

2	1	3	2	4	1	1	5

Kürzeste gefundene Teilsequenz: [3, 2, 4] · Länge 3

Sliding Windows – Kürzeste Teilsequenz mit Summe ≥ Val

- **Naiver Ansatz**: Für jede Fensterposition alle Werte erneut aufsummieren.
- **Und hier ist sie wieder … die Brute-Force-Implememngtierung mit zwei Schleifen ☺**

```
public static int minLengthSequenceSumBruteForce(int[] values, int target) {
    int minLength = Integer.MAX_VALUE;
    for (int start = 0; start < values.length; start++) {
        int sum = 0;
        for (int end = start; end < values.length; end++) {
            sum += values[end];
            if (sum >= target) {
                minLength = Math.min(minLength, end - start + 1);
                break; // Für diesen Start wird das Teilarray nicht kürzer.
            }
        }
    }
    return minLength == Integer.MAX_VALUE ? 0 : minLength;
}
```

Sliding Windows – Kürzeste Teilsequenz mit Summe ≥ Val

- **Sliding Window Variante**

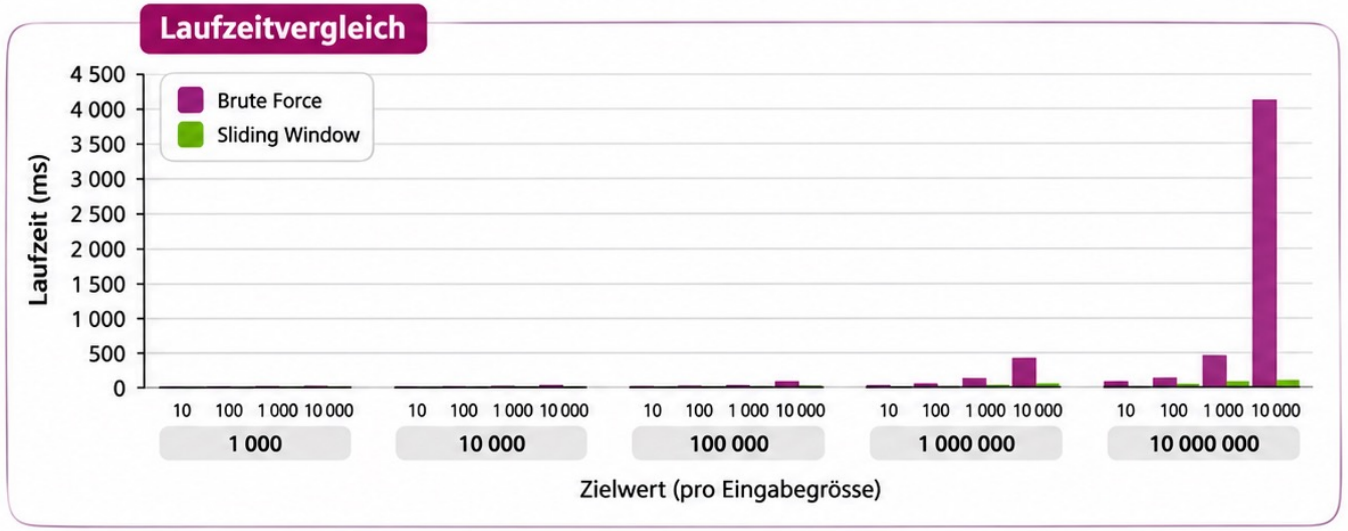
```
public static int minLengthSequenceSum(int[] values, long target) {
    int left = 0;
    int shortest = Integer.MAX_VALUE;
    long sum = 0;

    for (int right = 0; right < values.length; right++) {
        sum += values[right];
        while (sum >= target) {
            shortest = Math.min(shortest, right - left + 1);
            sum -= values[left++];
        }
    }
    return shortest == Integer.MAX_VALUE ? 0 : shortest;
}
```

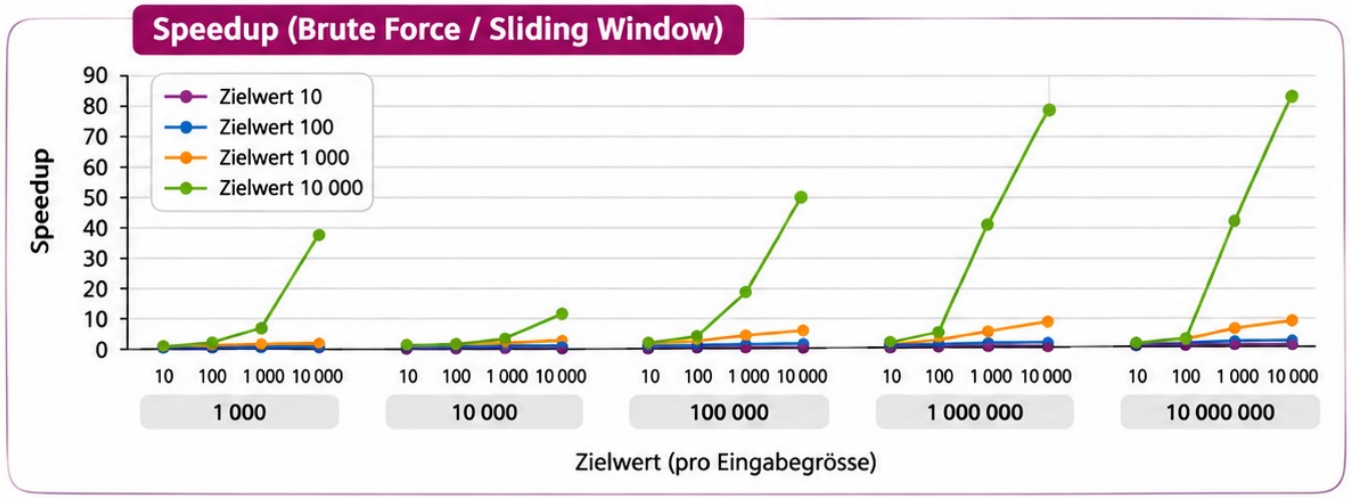
DEMO: `MinLengthSequenceTargetSumBenchmark.java`

Messungen – Kürzeste Teilsequenz mit Summe ≥ Val

Grösse	Zielwert	Brute Force (ms)	Sliding Window (ms)	Speedup
1'000	10	0.061	0.045	1.34×
	100	0.174	0.054	3.23×
	1'000	0.373	0.117	3.19×
	10'000	0.867	0.011	**78.48×**
10'000	10	0.125	0.159	0.79×
	100	0.129	0.107	1.21×
	1'000	0.558	0.158	3.53×
	10'000	4.412	0.420	**10.51×**
100'000	10	0.455	0.369	1.23×
	100	1.030	0.493	2.09×
	1'000	4.946	0.465	10.63×
	10'000	44.442	0.882	**50.37×**
1'000'000	10	5.494	6.427	0.85×
	100	9.910	5.186	1.91×
	1'000	46.235	5.691	8.12×
	10'000	414.694	5.348	**77.54×**
10'000'000	10	50.088	44.829	1.12×
	100	94.144	49.759	1.89×
	1'000	456.326	49.793	9.16×
	10'000	4'130.683	49.658	**83.18×**

Messungen – Kürzeste Teilsequenz mit Summe ≥ Val





Erkenntnisse

Deutlich bessere Skalierung: Sliding Window bleibt auch bei sehr grossen Eingaben schnell.

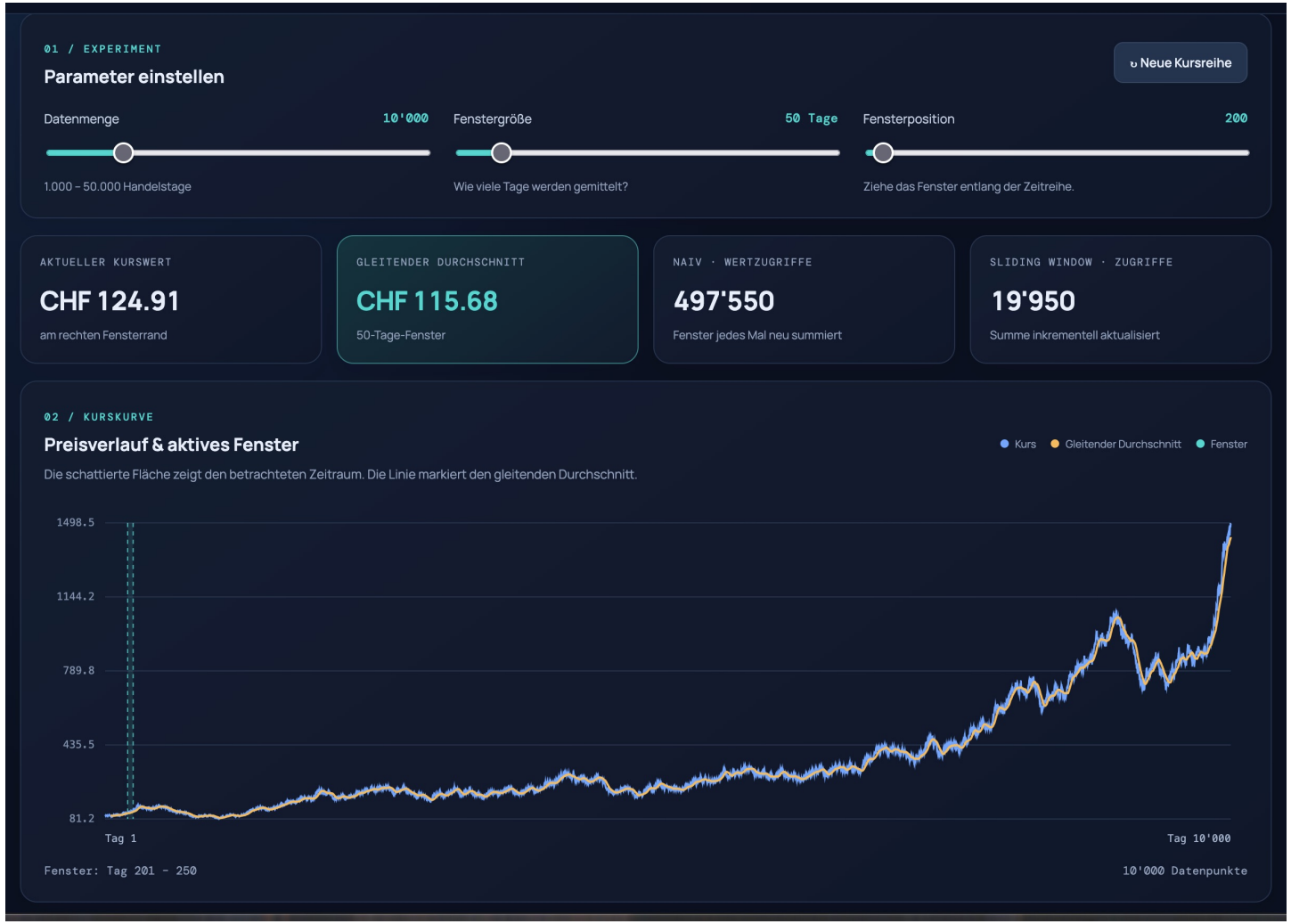
Grosser Vorteil bei hohen Zielwerten: Der Speedup steigt auf über **83×**.

Bei kleinen Zielwerten ähnlich schnell: Die Laufzeiten sind vergleichbar (Speedup ≈ 1×).

Fazit

Sliding Window ist für grosse Eingaben und hohe Zielwerte deutlich effizienter und deshalb die bessere Wahl.

DEMO: `Sliding Window Arena.java`



Exercises

🧩 Rekursion und Memoization

Rekursion

- Rekursion ist eine Vorgehensweise, bei der eine Funktionalität sich selbst aufruft.
- Klingt merkwürdig, führt aber mitunter zu sehr eleganter Lösungsfindung.
- Möglichkeit, komplizierte Probleme in einfachere Teilprobleme zu zerlegen, die leichter zu lösen sind:

Abbruchbedingung («Verankerung»)

$$factorial(n) = \begin{cases} 1 & (n = 0) \\ n \cdot factorial(n-1) & (n > 0) \end{cases}$$

Funktion verwendet sich selbst («Induktionsschritt»)

IDE: RecursiveFactorial
IDE: InfiniteRecursion

Rekursion und ihre Beschränkungen / Überraschungen

```
long factorial(int number) {
    if (number == 0) {
        return 1;
    } else {
        return number * factorial(number - 1);
    }
}
```

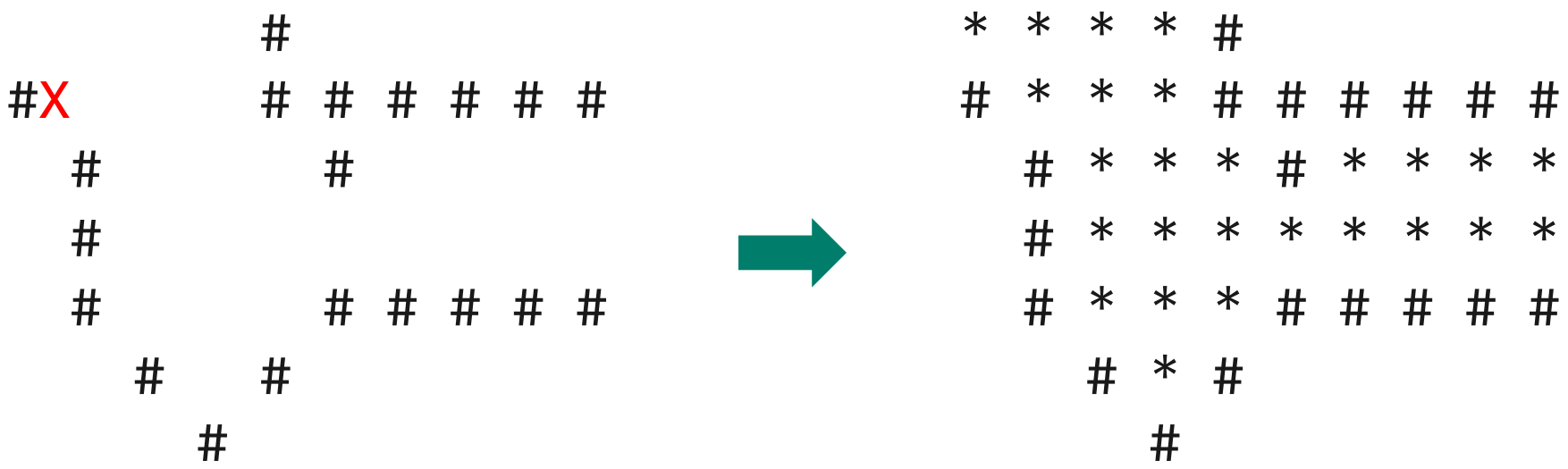
```
IO.println(factorial(7));
IO.println(factorial(14));
IO.println(factorial(77));
```

```
5040
87178291200
0
```

Praktische Anwendungsfälle von Rekursion

IDE: FloodFillExample

- Füllen von Flächen mit Muster (**Flood-Fill**)



- **Lösen** von **Sudoku**-Rätseln, **Acht-Damen-Problem**, **Weg des Springers** etc.
- **Traversierungen** von Bäumen

Reale Anwendungsfälle

IDE: FloodFillExampleWithPattern

- Verbesserung beim Füllen von Flächen mit Muster (Pattern-Flood-Fill)

```
## o                    #
#  ##################   ##
  #         ##          ###
  #         ##           ##
#  ################      #
##                       ##
###                     ###
##                       ##
#                        #
```

```
##..|..|..|..|..|..|..|..#
#*-###################--*-##
.|#         ##|..|..|..|###
-*#         ##*--*--*--*-##
#|.################|..|..#
##--*--*--*--*--*--*--*-##
###.|..|..|..|..|..|..|###
##--*--*--*--*--*--*--*-##
#|..|..|..|..|..|..|..|..#
```

Was meint ihr, wie aufwändig das Ganze ist?
Wie lange brauchen wir dafür? T-Shirt-Size S, M, L?

Berechnung der Fibonacci-Zahlen

- Die Fibonacci-Zahlen sind rekursiv wie folgt definiert:

$$fib(n) = \begin{cases} 1, & n = 1 \\ 1, & n = 2 \\ fib(n-1) + fib(n-2), & \forall n > 2 \end{cases}$$

- Damit ergeben sich folgende Werte:

n	1	2	3	4	5	6	7	8
fib(n)	1	1	2	3	5	8	13	21

Berechnung der Fibonacci-Zahlen: Problemstellung

IDE: Fibonacci

- Analog zur mathematischen Formel erfolgt die rekursive Implementierung:

```
static long fibRec(int n) {
    if (n <= 0)
        throw new IllegalArgumentException("n must be >= 1");

    if (n == 1 || n == 2)
        return 1;

    // rekursiver Abstieg
    return fibRec(n - 1) + fibRec(n - 2);
}
```

- Diese Lösung ist zwar sehr elegant, da sie die mathematische Definition nahezu 1:1 widerspiegelt, und zudem leicht nachvollziehbar und sehr verständlich ist.

- So schön diese Implementierung auch von der Programmierung her ist, birgt sie doch einen Fallstrick: Sie ist nämlich nicht besonders effizient.

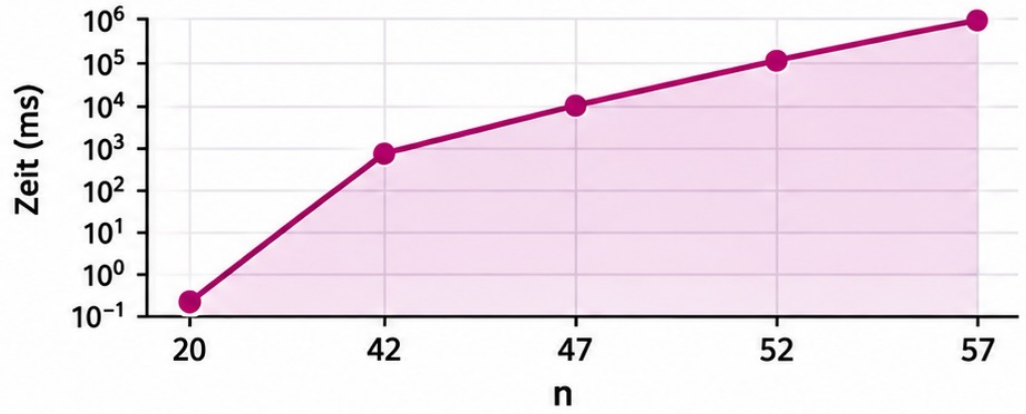
Berechnung der Fibonacci-Zahlen: Erkenntnisse

Fibonacci – Rekursive Berechnung

Messergebnisse

n	Ergebnis	Zeit (ms)
20	6’765	0.142
42	267’914’296	456.266
47	2’971’215’073	4’606.407
52	32’951’280’099	51’370.009
57	365’435’296’162	583’443.623

Laufzeit



Erkenntnisse

Exponentielles Wachstum

Die Laufzeit steigt bei grösserem n sehr stark an.

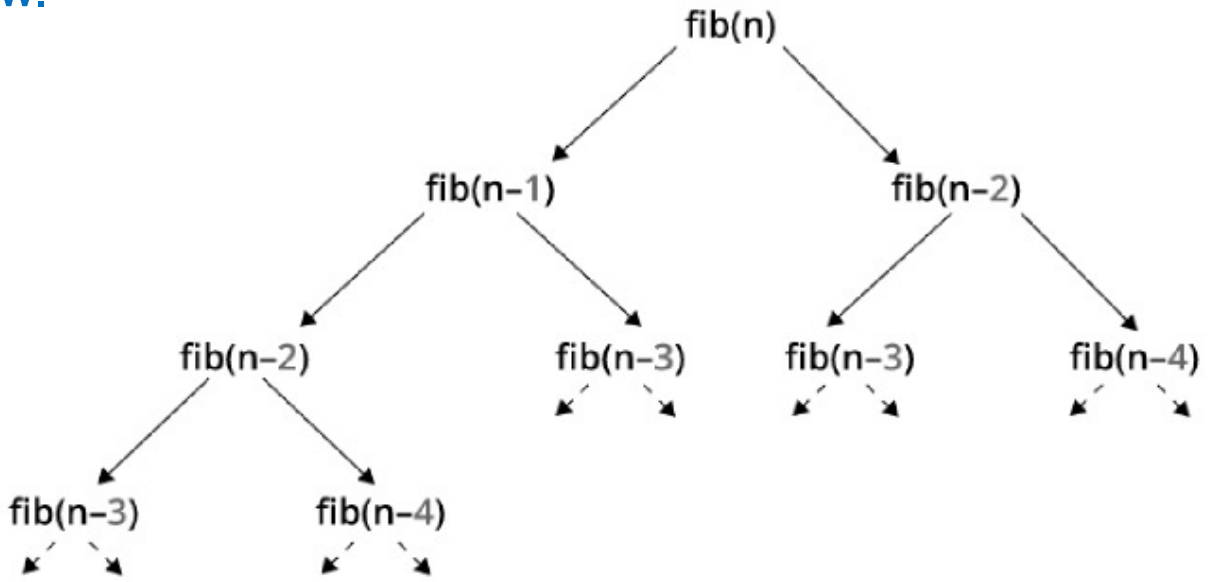
Fazit

Die rekursive Fibonacci-Berechnung ist für grössere n sehr ineffizient und in der Praxis nicht geeignet.

Was ist der Grund?

Berechnung der Fibonacci-Zahlen

- Es werden immer wieder die gleichen Fibonacci-Zahlen berechnet, etwa mehrmals `fib(n - 2)` oder `fib(n - 3)` usw.



- **Grund für die lange Laufzeit sind die eigentlich überflüssigen Aufrufe für zuvor schon durchgeführten Berechnungen.**

Wie kann man diese Aufrufe verhindern? Vorschläge?
Tipp: Wir kennen dafür schon etwas …

Berechnung der Fibonacci-Zahlen mit Memoization

- Die Lösung liegt in einem Cache/Lookup-Map. Für Rekursion spricht man von Memoization.
- Das steht für das Zwischenspeichern von Berechnungsresultaten.
- Dafür kann man eine simple Liste nutzen, aber oftmals ist eine Map geeigneter, in diesem Fall, um dort alle bereits berechneten Fibonacci-Zahlen abzuspeichern:

```
static long fibonacciOptimized(int n) {
    if (n <= 0)
        throw new IllegalArgumentException("n must be >= 1");

    return fibonacciMemo(n, new HashMap<>());
}
```

Berechnung der Fibonacci-Zahlen mit Memoization

- Bevor wir eine Berechnung vornehmen, prüfen wir die Existenz eines passenden Berechnungsresultats in der Map.
- Nur bei Nichtexistenz erfolgt dann die Berechnung und anschließend die Ablage in der Map.

```
static long fibonacciMemo(int n, Map<Integer, Long> lookupMap) {
    if (lookupMap.containsKey(n)) // MEMOIZATION: prüfe, ob Ergebnis existiert
        return lookupMap.get(n);

    // normaler Algorithmus mit Hilfsvariable für Resultat
    if (n == 1 || n == 2)
        return 1;

    long result = fibonacciMemo(n - 1, lookupMap) + fibonacci_memo(n - 2, lookupMap);
    lookupMap.put(n, result); // MEMOIZATION: speichere berechnetes Ergebnis

    return result;
}
```

Fibonacci-Zahlen: Berechnung-Trick

IDE: ListFibonacciTrickExample

- Man kann das Ganze sogar noch trickreicher wie folgt implementieren, hier werden vorberechnete Werte wiederverwendet. Man spricht von Tabulation (Dynamic Proramming)

```
jshell> var fib_numbers = new ArrayList<>(List.of(0, 1))
fib_numbers ==> [0, 1]

jshell> for (int i = 0; i < 10; i++) {
   ...>     int lastIdx = fib_numbers.size();
   ...>     fib_numbers.add(fib_numbers.get(lastIdx - 2) +
   ...>                     fib_numbers.get(lastIdx - 1));
   ...> }

jshell> fib_numbers
fib_numbers ==> [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Ausblick: Dynamic Programming

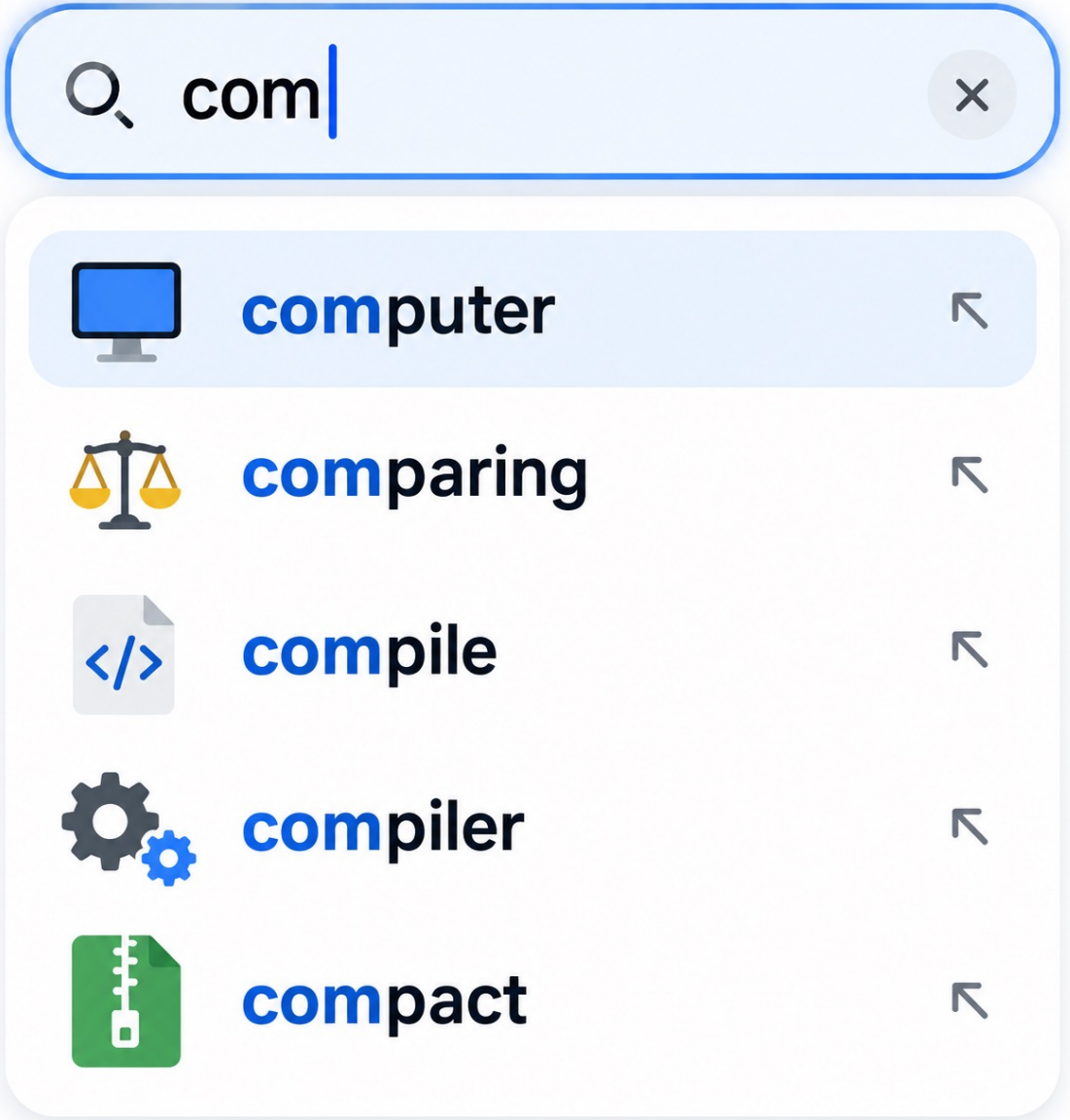
- **Grundidee**: Komplexe Probleme werden in kleinere Teilprobleme zerlegt.
- **Wiederverwendung**: Bereits berechnete Ergebnisse werden gespeichert, um redundante Berechnungen zu vermeiden.
- **Zwei Ansätze**: Memoization (Top-down) und Tabulation (Bottom-up).
- **Voraussetzung**: Überlappende Teilprobleme und eine optimale Teilstruktur.
- **Typische Anwendungen**: Kürzeste Wege, Rucksackproblem, Münzwechselproblem und Edit Distance.
- **Effizienzgewinn**: Häufig lässt sich exponentielle Laufzeit auf polynomielle Laufzeit reduzieren.

Nicht immer müssen wir schneller rechnen – manchmal
reicht es, bereits Berechnetes nicht erneut zu berechnen.

DEMO: “additional_examples”

Exercises

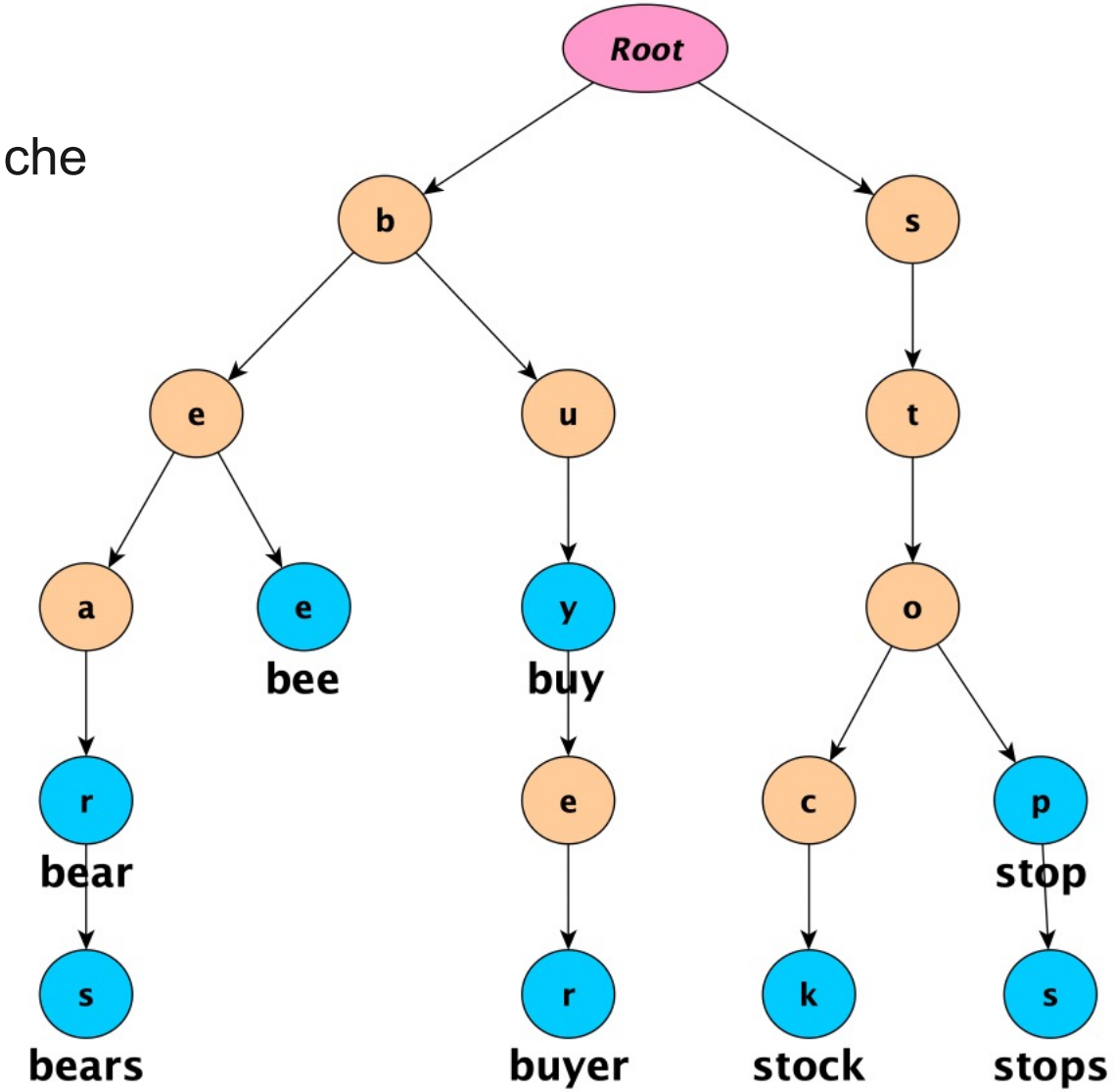
🌳 Tries – Auto Complete



Große und zentrale Frage: Wie verwalten und finden wir alle Wörter, die mit einem bestimmten Präfix beginnen, ohne jedes Wort vollständig zu prüfen?

Die Datenstruktur namens Trie

- Die Datentstruktur **Trie** ermöglicht eine effiziente Suche nach gespeicherten Wörtern und Präfixen.
- Ein **Trie** ist ein **Baum**, in dem jeder **Pfad** ein **Präfix** repräsentiert.
- Die **Kanten** stehen für **Zeichen**, und **Knoten** repräsentieren **Buchstaben** markieren gegebenenfalls das Ende eines vollständigen Wortes (hier blau gekennzeichnet).



Die Klasse `Trie`

- Die Klasse `Trie` bildet den zentralen Einstiegspunkt für alle Operationen.
- Das Attribut root speichert die Wurzel des `Trie`.
- Die Wurzel repräsentiert keinen Buchstaben, sondern dient als Ausgangspunkt für Einfügen, Suchen, Autocomplete und Löschen.

```
public class Trie {
    private final TrieNode root = new TrieNode(null);

    public void insert(String word) {
        // …
    }

    public boolean contains(String word) {
        // …
    }

    // …
}
```

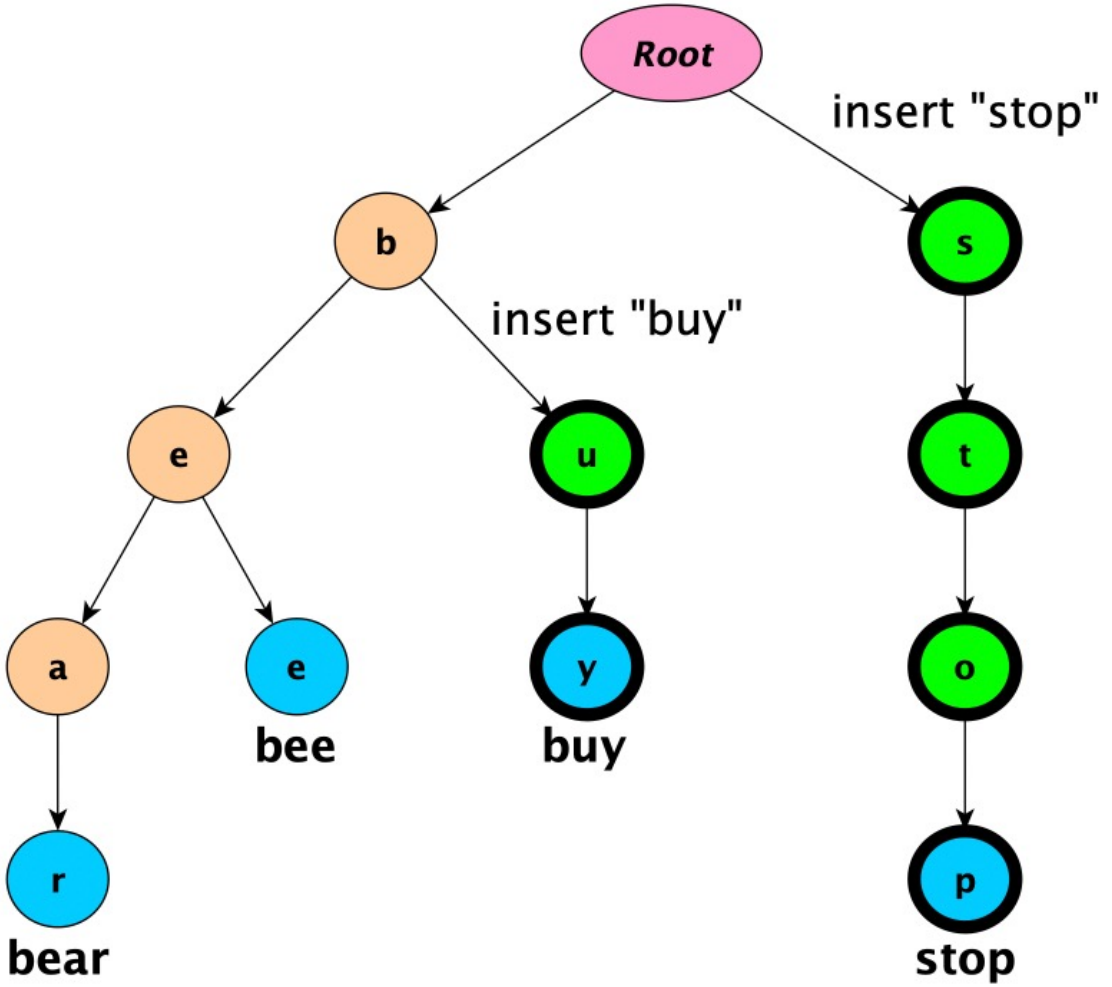
Die Klasse `TrieNode`

- Jeder Buchstabe wird durch eine eigene Instanz von `TrieNode` repräsentiert.
- Eine `Map` verknüpft Buchstaben (Key) mit den zugehörigen `TrieNode`-Instanzen (Value).
- Das Attribut `ch` speichert den Buchstaben zusätzlich.
- Das Attribut `word` kennzeichnet das Ende eines vollständigen Wortes.

```
public class TrieNode {
    final Map<Character, TrieNode> children = new HashMap<>();
    final Character ch;
    String word = null;

    // …

    public boolean isWord() {
        return word != null;
    }
}
```

Wort einfügen

- Wort wird Buchstabe für Buchstabe eingefügt, beginnend beim Root-Knoten.
- Für jeden Buchstaben wird geprüft, ob bereits eine passende `TrieNode` existiert.
  - Falls nicht, wird eine neue `TrieNode` angelegt.
- Das wiederholt sich bis zum letzten Buchstaben.
- Das Attribut `word` speichert vollständiges Wort.



Wort einfügen

- Wort wird Buchstabe für Buchstabe eingefügt, beginnend beim Root-Knoten.
- Für jeden Buchstaben wird geprüft, ob bereits eine passende `TrieNode` existiert.
  - Falls nicht, wird eine neue `TrieNode` angelegt.
- Das wiederholt sich bis zum letzten Buchstaben.
- Das Attribut `word` speichert vollständiges Wort.

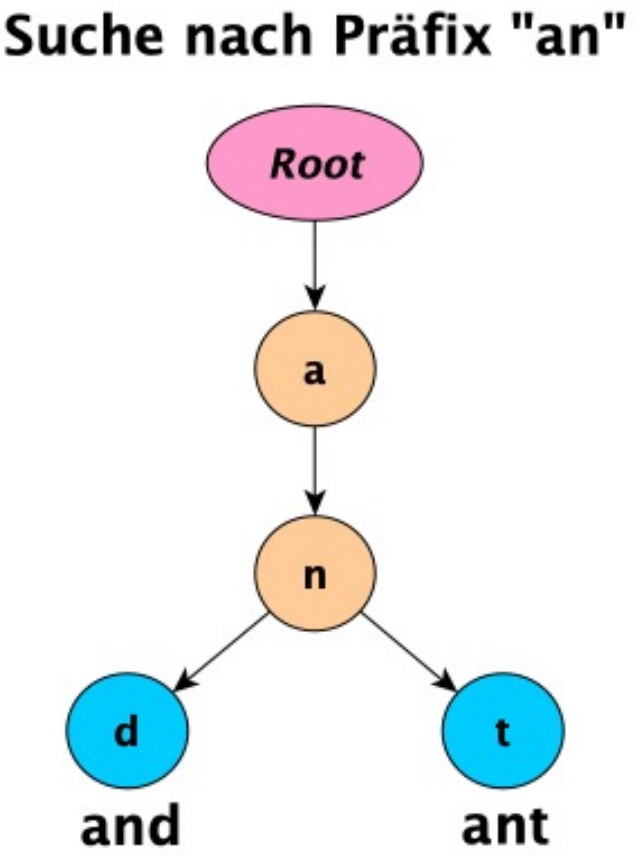
```
public void insert(String word) {
    TrieNode current = root;
    for (char ch : word.toCharArray()) {
        current.createChildIfAbsent(ch);
        current = current.childFor(ch);
    }
    current.word = word;
}
```

Wort auf Enthaltensein prüfen

- Die Suche erfolgt Buchstabe für Buchstabe vom Root-Knoten aus.
- Das Vorgehen ähnelt dem Einfügen eines Wortes.

```
public boolean contains(String word) {
    TrieNode current = findNodeForCharSequence(word);
    if (current == null)
        return false;

    return current.isWord();
}

private boolean containsCharSequence(String prefix) {
    return findNodeForCharSequence(prefix) != null;
}
```

- Verfahren basiert auf der Hilfsmethode `findNodeForCharSequence()`.



Wort prüfen auf Enthaltensein – Hilfsmethode

- Die Methode `findNodeForCharSequence()` kapselt die Navigation im Trie.
- Sie kann für die Prüfung von vollständigen Wörtern und Präfixen wiederverwendet werden.

```
private TrieNode findNodeForCharSequence(String prefix) {
    TrieNode current = root;

    for (char ch : prefix.toCharArray()) {
        current = current.childFor(ch);
        if  (current == null)
            return null;
    }

    return current;
}
```

Vorschäge generieren – Auto Complete

- Autocomplete generiert Wortvorschläge basierend auf einem eingegebenen Präfix.
- `findNodeForCharSequence()` sucht die passende `TrieNode` zum Präfix.
- Wird kein passender Knoten gefunden, ist die Vorschlagsliste leer.
- Andernfalls übernimmt `complete()` in `TrieNode` die Generierung der Vorschläge.
- Die Klasse `Trie` steuert den Ablauf der Autocomplete-Funktion.

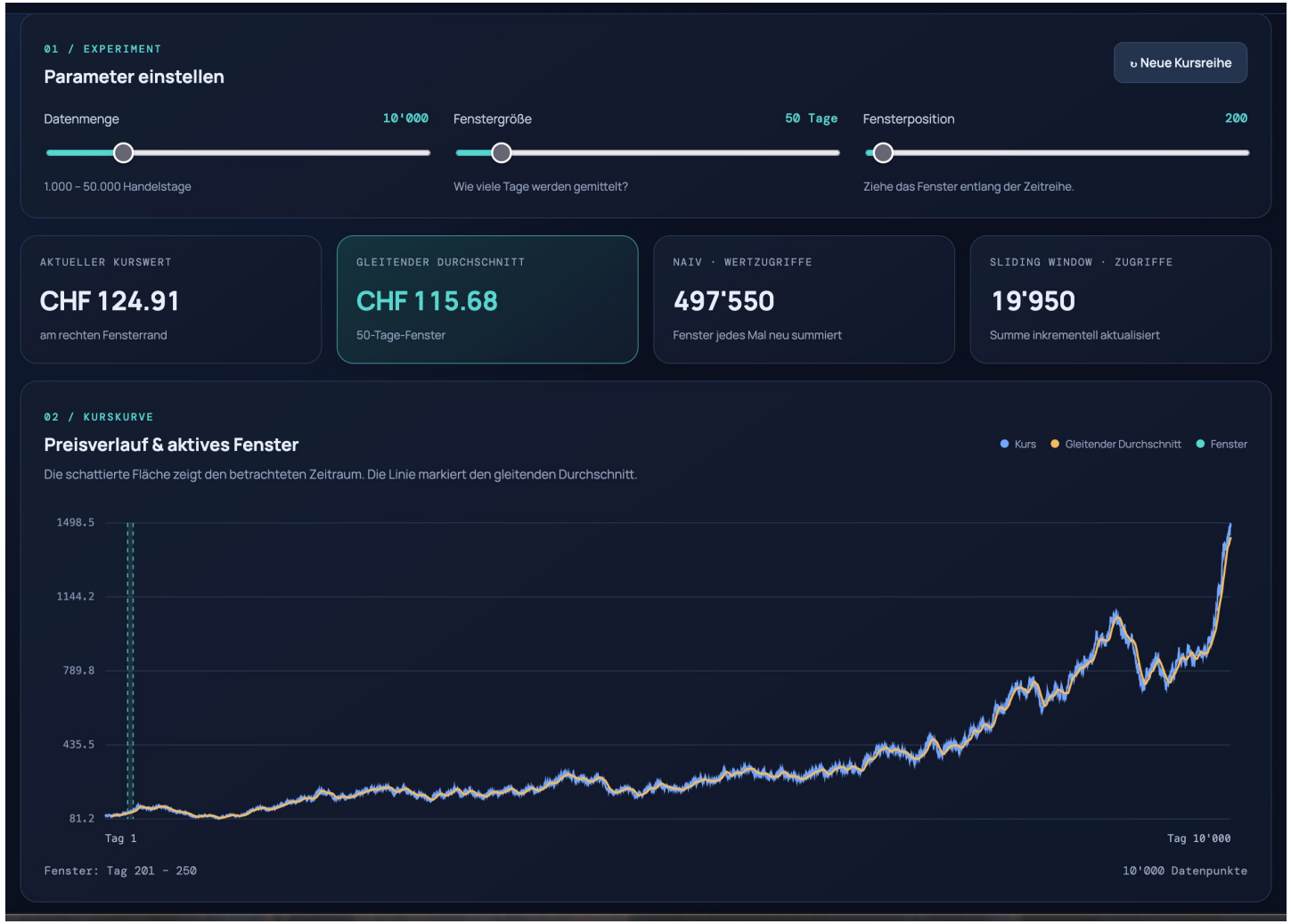
```
public List<String> complete(String prefix) {
    TrieNode current = findNodeForCharSequence(prefix);
    if  (current == null)
        return List.of();

    return current.complete();
}
```

Vorschäge generieren – Auto Complete

- Rekursive Implementierung der Methode `complete()` in `TrieNode`.
- Prüfung auf vollständige Wörter und Übernahme in die Ergebnisliste.
- Rekursives Durchsuchen aller Folgeknoten zur Generierung der Vorschläge.

```
List<String> complete() {
    List<String> result = new ArrayList<>();
    if (isWord())
        result.add(word);

    for (TrieNode child : children.values()) {
        result.addAll(child.complete());
    }

    return result;
}
```

DEMO: `Trie Arena.java`



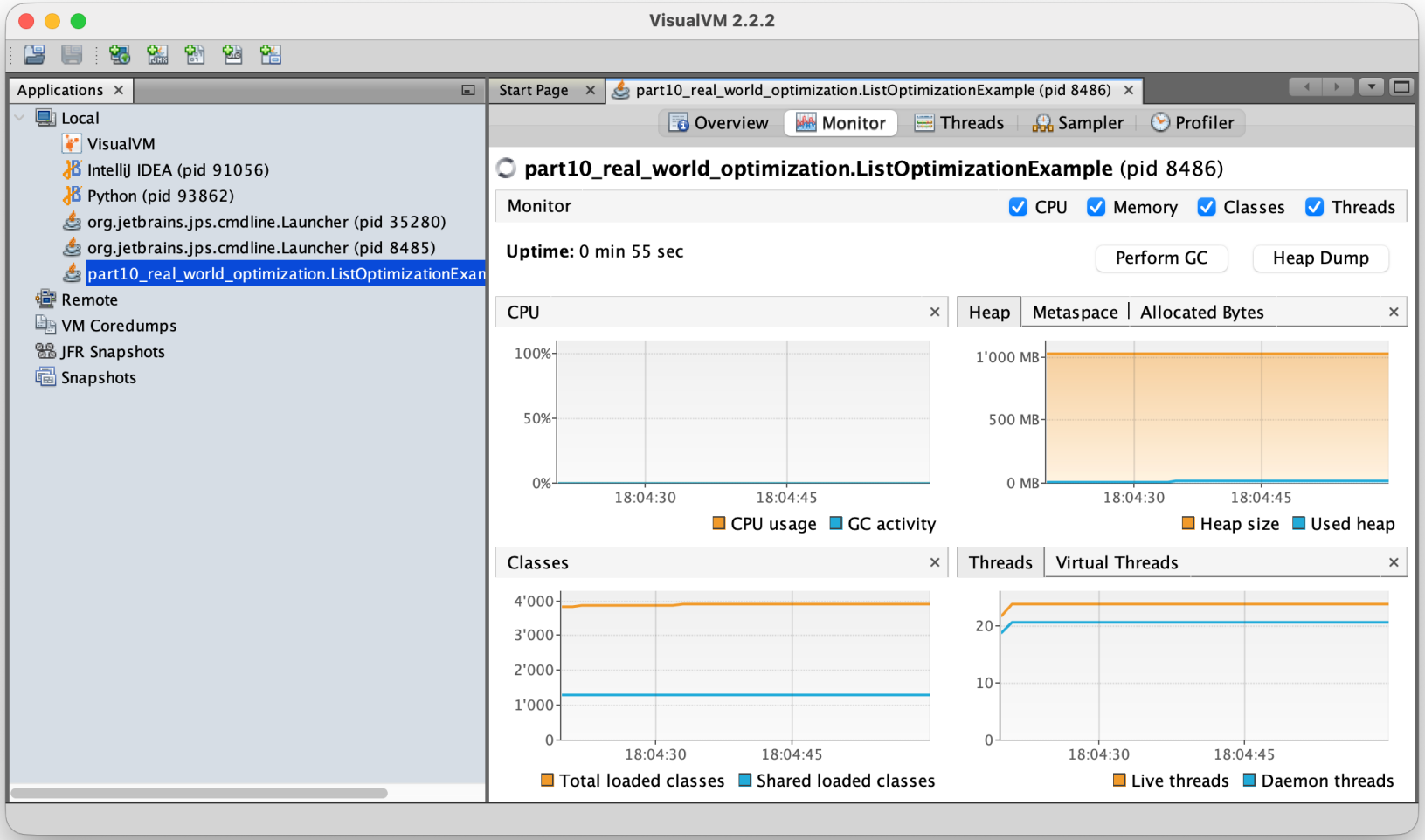
Exercises

🏁 Performance-Finale

Messen, erklären, optimieren mit Visual VM

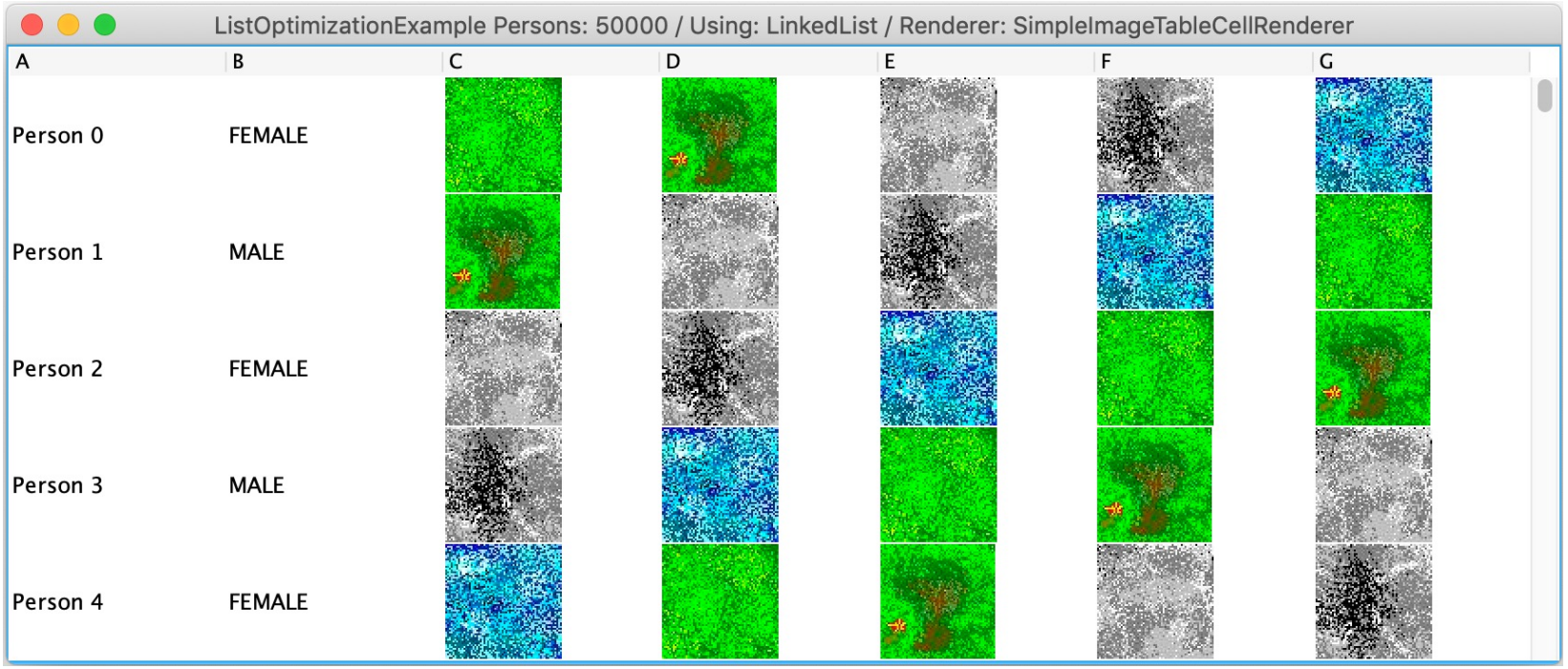
Visual VM

- Einsatz von VisualVM zur Analyse und Identifikation von Performance-Problemen.



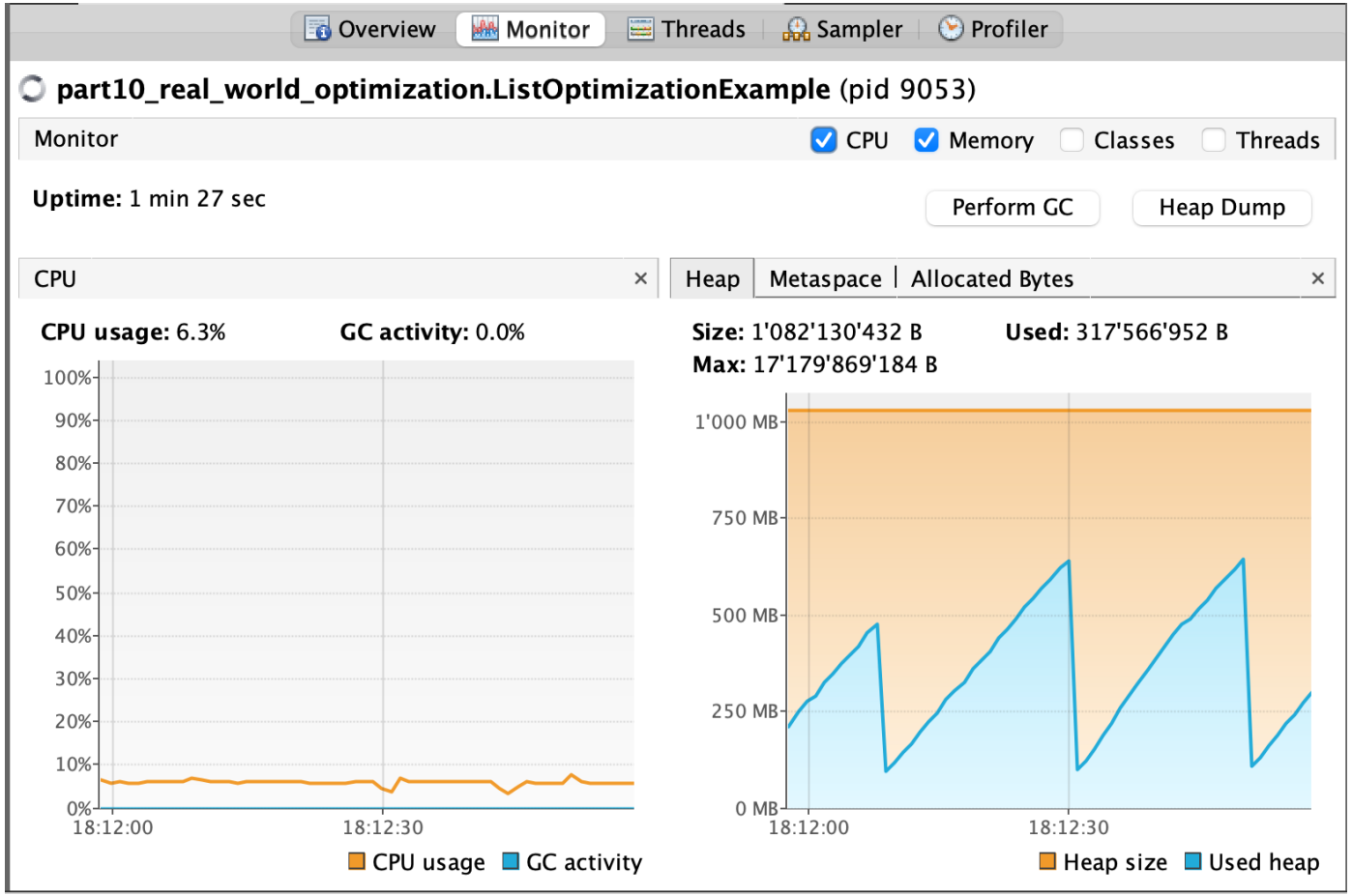
Performance-Finale

- Einsatz von VisualVM zur Analyse und Identifikation von Performance-Problemen.
- Praxisbeispiel: Optimierung einer langsamen Tabellenkomponente mit vielen Einträgen.
- Ziel: Performance-Engpässe erkennen und gezielt optimieren.

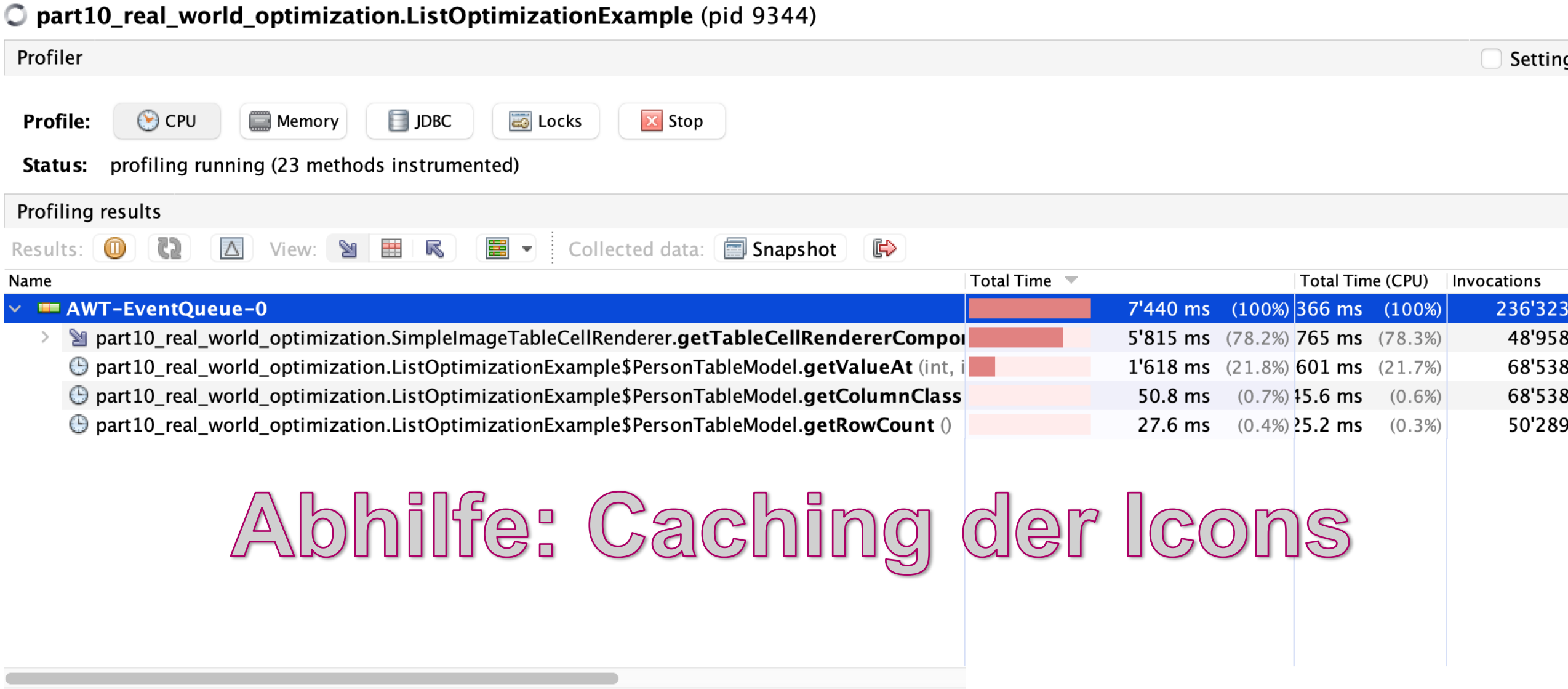


Performance-Finale

- Ausgangsbasis:
  - Grafiken werden bei jedem Zeichenvorgang erneut aus dem Dateisystem geladen.
  - Datenhaltung basiert auf einer `LinkedList`

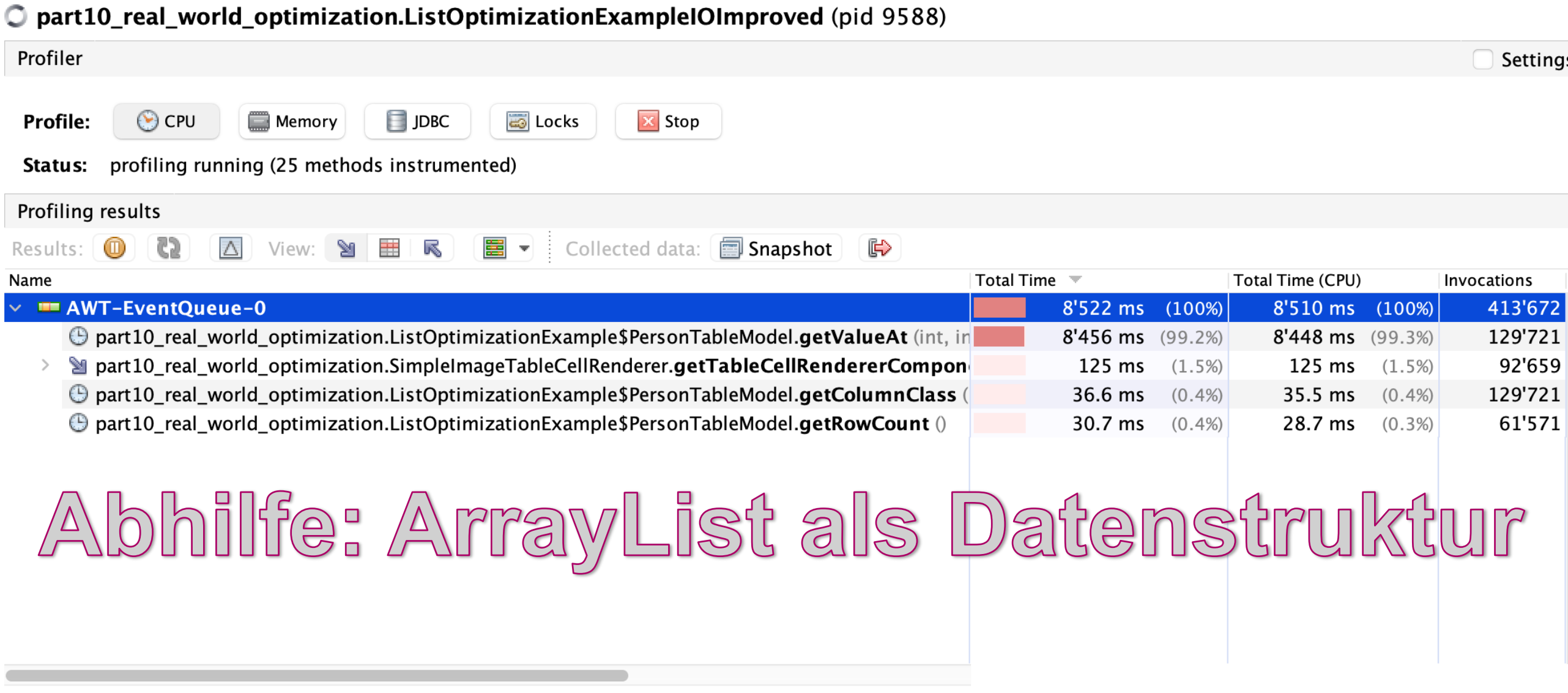


Profiling: `ListOptimizationExample`



Abhilfe: Caching der Icons

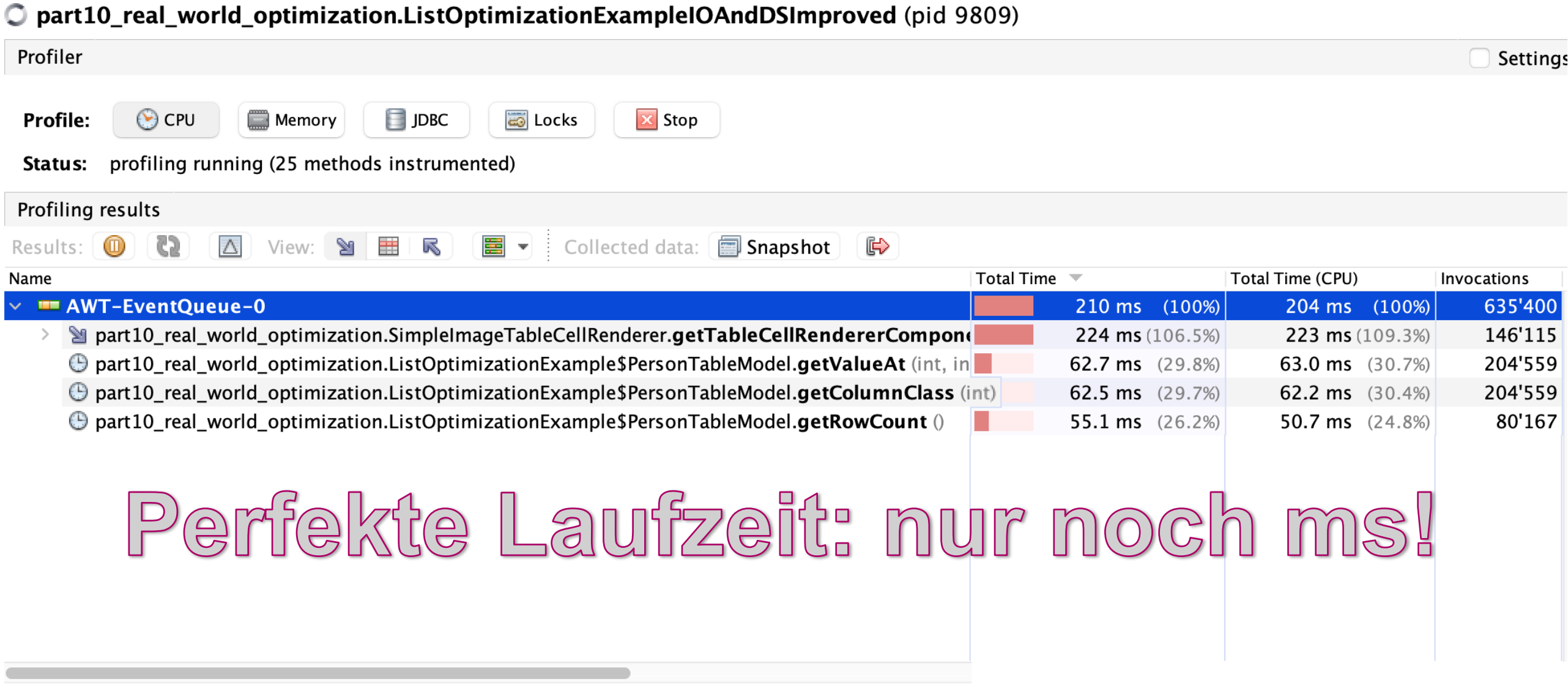
Profiling: `ListOptimizationExampleIOImproved`



Abhilfe: ArrayList als Datenstruktur

🏁 Performance-Finale

Profiling: `ListOptimizationExampleIOAndDSImproved`

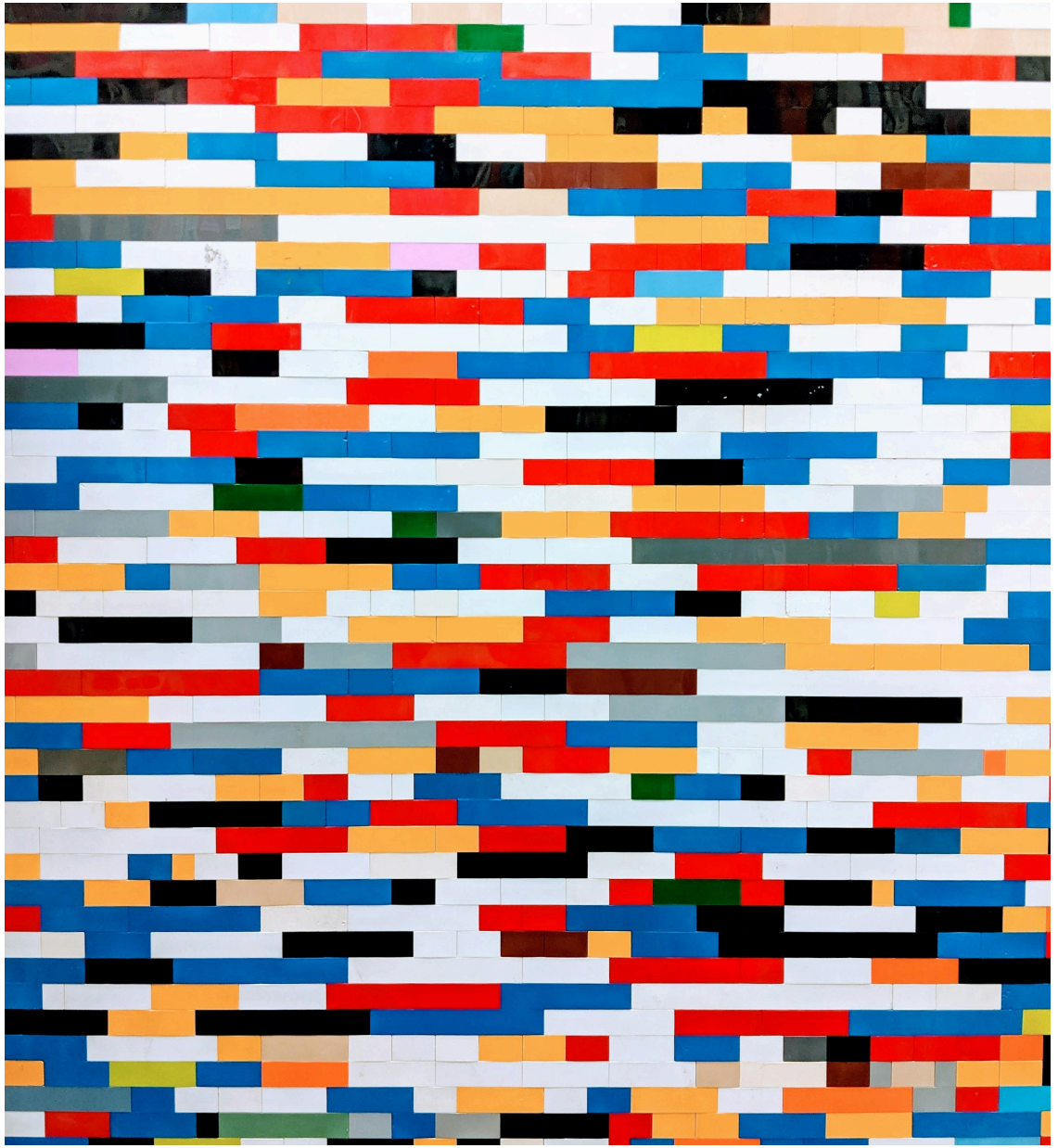


Perfekte Laufzeit: nur noch ms!

Exercises

🔥 Performance Fireworks

“Wir wollen Laufzeitkomplexitäten nun gezielt reduzieren – etwa von quadratisch zu linear oder sogar konstant.“



Datastrukturen- und Methoden-Fallstrick: O(n²) → O(n)

- **Bekanntes mitunter problematisches Muster:**

```
String result = "";
for (String item : data) {
    result = result + transform(item);
}
```

Wirkt wie O(n)! Kann aber O(n²) sein!

```
1 element → copy 1
2 elements → copy 2
...
n elements → copy n
=> 1 + 2 + 3 + ... + n = O(n²)
```

- **Der Algorithmus is nicht das Problem — die ungünstige Art der Verwendung der Datenstruktur (hier String) führt zu unnötigen Kopiervorgängen.**

```
StringBuilder result2 = new StringBuilder();
for (String item : data) {
    result2.append(transform(item));
}
```

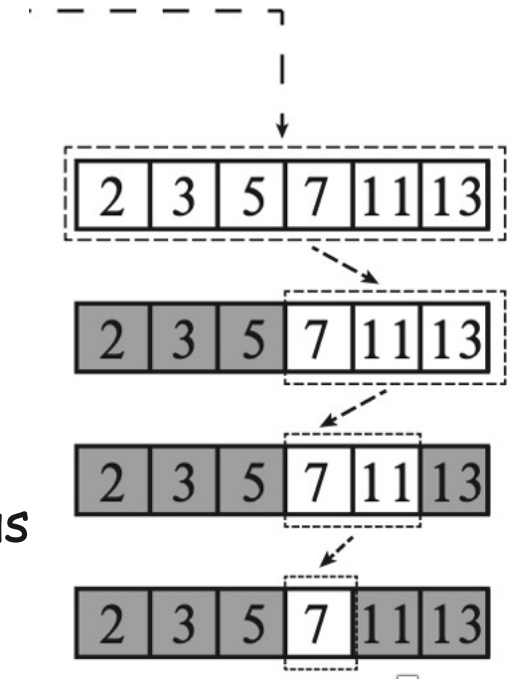
Hinweis: Nicht jede String-Verkettung ist $O(n^2)$. Problematisch ist das in Schleifen, da dabei wiederholt neue Strings mit dem bisherigen Inhalt entstehen.

Algorithmus: Lineare vs. binäre Suche: O(n) → O(log n)

- **Lineare Suche, sofern keine Zusatzinfo über Daten:**

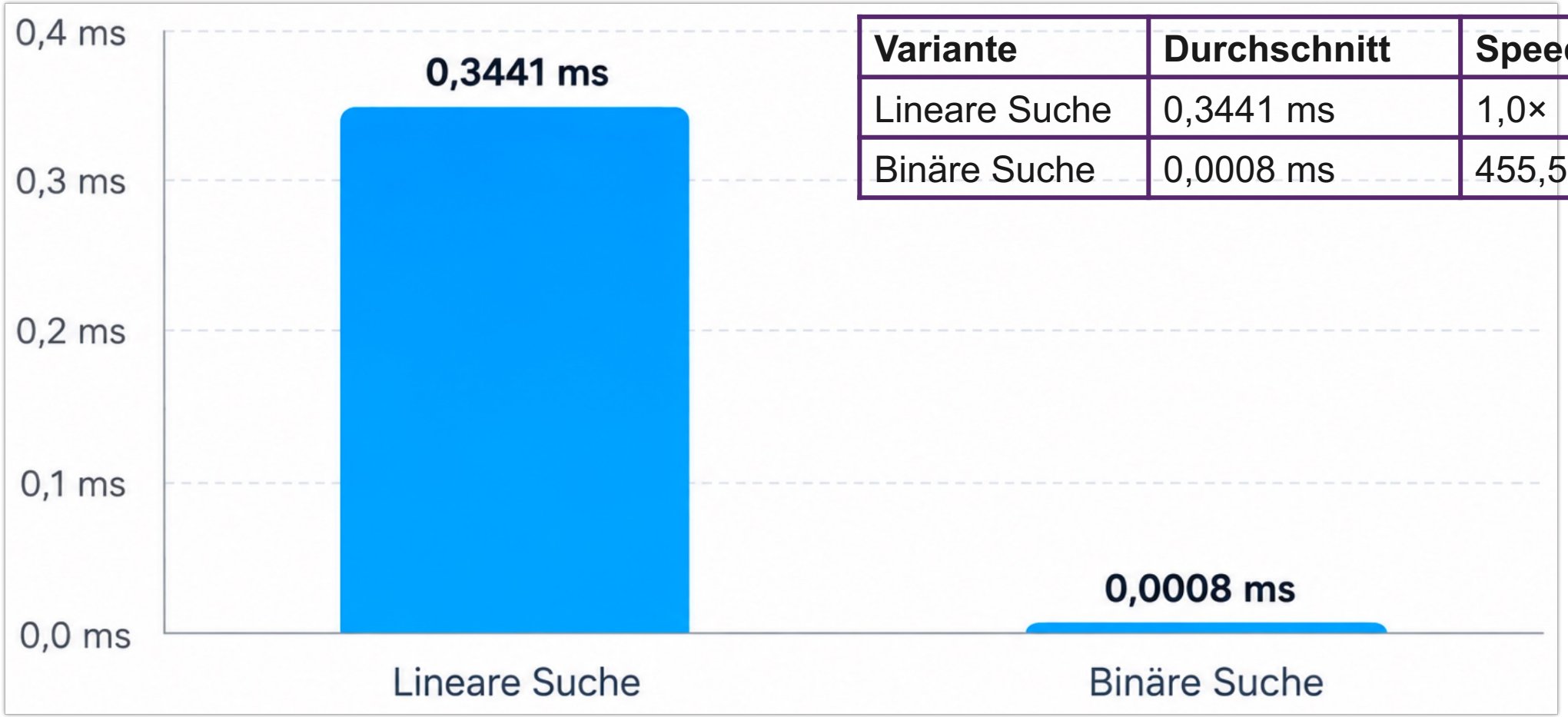
```
int linearSearch(List<Long> values, long target)
{
    for (int i = 0; i < values.size(); i++) {
        if (values.get(i).equals(target))
            return i;

    return -1;
}
```

- **Binärsuche, falls wir wissen, die Daten sind sortiert**
  - Die Hälfte der Daten ausschliessen auf dem Weg zum Ziel



Takeaway: Manchmal liegt das größte Optimierungspotenzial darin, einen anderen Algorithmus zu wählen. Erst Algorithmus hinterfragen, bevor man die Implementierung optimiert.

Performance: Lineare vs binäre Suche: O(n) → O(log n)



Variante	Durchschnitt	Speed-up vs naiv
Lineare Suche	0,3441 ms	1,0×
Binäre Suche	0,0008 ms	455,57×

Algorithmus: Primzahlen ≤ n: $O(n^2) \rightarrow O(n \sqrt{n}) = O(n^{1.5})$

```
static boolean isPrime(int n) {
    if (n < 2)
        return false;

    for (int d = 2; d < n; d++) {
        if (n % d == 0)
            return false;
    }
    return true;
}
static List<Integer> naiveCalcPrimes(int n) {
    List<Integer> primes = new ArrayList<>();
    for (int i = 2; i <= n; i++) {
        if (isPrime(i))
            primes.add(i);
    }
    return primes;
}
```

Obere Hälfte irrelevant → n/2

```
for (int d = 2; d <= n // 2; d++)
```

Nur bis √n prüfen

```
for (int d = 2; d * d <= n ; d++)
```

"Der Code ist schneller – aber es ist derselbe Algorithmus."

Noch besser: "Wir müssen nur bis √n prüfen"

Takeaway

Nach einem besser geeigneten mathematischen Modell suchen.

Algorithmus: Primzahlen: O(n √n) → O(n log log n)

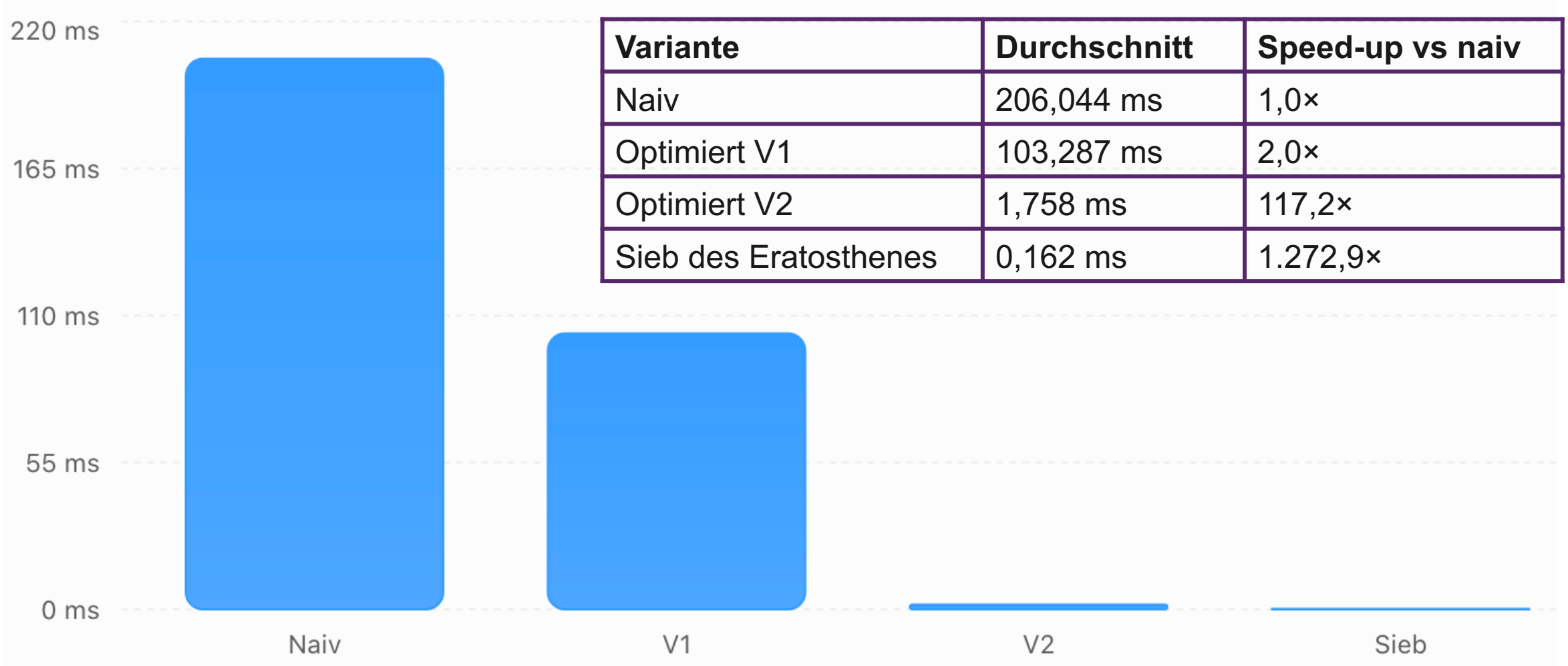
- **Perspektivenwechsel**: “Warum jede Zahl einzeln auf Primzahl testen? Stattdessen streichen wir Vielfache bekannter Primzahlen.”

$$2, 3, \cancel{4}, 5, \cancel{6}, 7, \cancel{8}, \cancel{9}, \cancel{10}, 11, \cancel{12}, 13, \cancel{14}, \cancel{15}$$

```
List<Integer> sieveOfEratosthenes(int n) {
    boolean[] isPrime = new boolean[n + 1];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;
    for (int p = 2; p * p <= n; p++) {
        if (isPrime[p]) {
            for (int multiple = p * p; multiple <= n; multiple += p)
                isPrime[multiple] = false;
        }
    }
    List<Integer> primes = new ArrayList<>();
    for (int i = 2; i <= n; i++)
         if (isPrime[i]) primes.add(i);
    return primes;
}
```

Takeaway
Das Sieb des Eratosthenes verhält sich ***nahezu linear*** in der Praxis: $O(n \log \log n)$.

Performance: Primzahlen: $O(n^2) \rightarrow O(n \log \log n)$



Variante	Durchschnitt	Speed-up vs naiv
Naiv	206,044 ms	1,0×
Optimiert V1	103,287 ms	2,0×
Optimiert V2	1,758 ms	117,2×
Sieb des Eratosthenes	0,162 ms	1.272,9×

Algorithmus: Summen-Schleife vs. Formel: O(n) → O(1)

- Die Aufgabe: Berechne die Summe der ersten n nicht-negativen ganzen Zahlen: $\sum_{i=0}^{n-1} i$

- **Erste scheinbar "gute" Implementierung – aber ist die Schleife wirklich nötig?**

```
sum = 0
for i in range(n):
    sum += i
```

„Don't optimize the loop. Remove the loop."

- **Algorithmus ist problematisch — durch pure Mathematik ersetzen:** $\text{sum} = \frac{n(n-1)}{2}$

```
sum = n * (n - 1) // 2
```

Kernaussage
Nicht nur den Code optimieren – zuerst prüfen, ob sich der Algorithmus grundsätzlich vereinfachen lässt.

Caching Tricks: Rekursion & Memoization O(2ⁿ) → O(n)

- **Naive Fibonacci-Berechnung**: einfache, elegante Rekursion – aber **extrem langsam**!
- Das Original wirkt harmlos, berechnet jedoch dieselben Zwischenwerte exponentiell oft erneut.

```
static long fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

```
static long fibMemo(int n, Map<Integer, Long> cache) {
    if (n <= 1)
        return n;

    if (cache.containsKey(n))
        return cache.get(n);

    long result = fibMemo(n - 1, cache) +
                  fibMemo(n - 2, cache);
    cache.put(n, result);
    return result;
}

static long fibMemo(int n) {
    return fibMemo(n, new HashMap<>());
}
```

Takeaway
Gleicher Lösungsansatz – aber durch Zwischenspeichern bereits berechneter Ergebnisse dramatisch schneller.

Recursion, Memoization, Formula: O(2ⁿ) → O(n) → O(1)

- Es gibt sogar eine mathematische Formel für die Fibonacci-Folge, deren Berechnung nocj deutlich schneller ist.

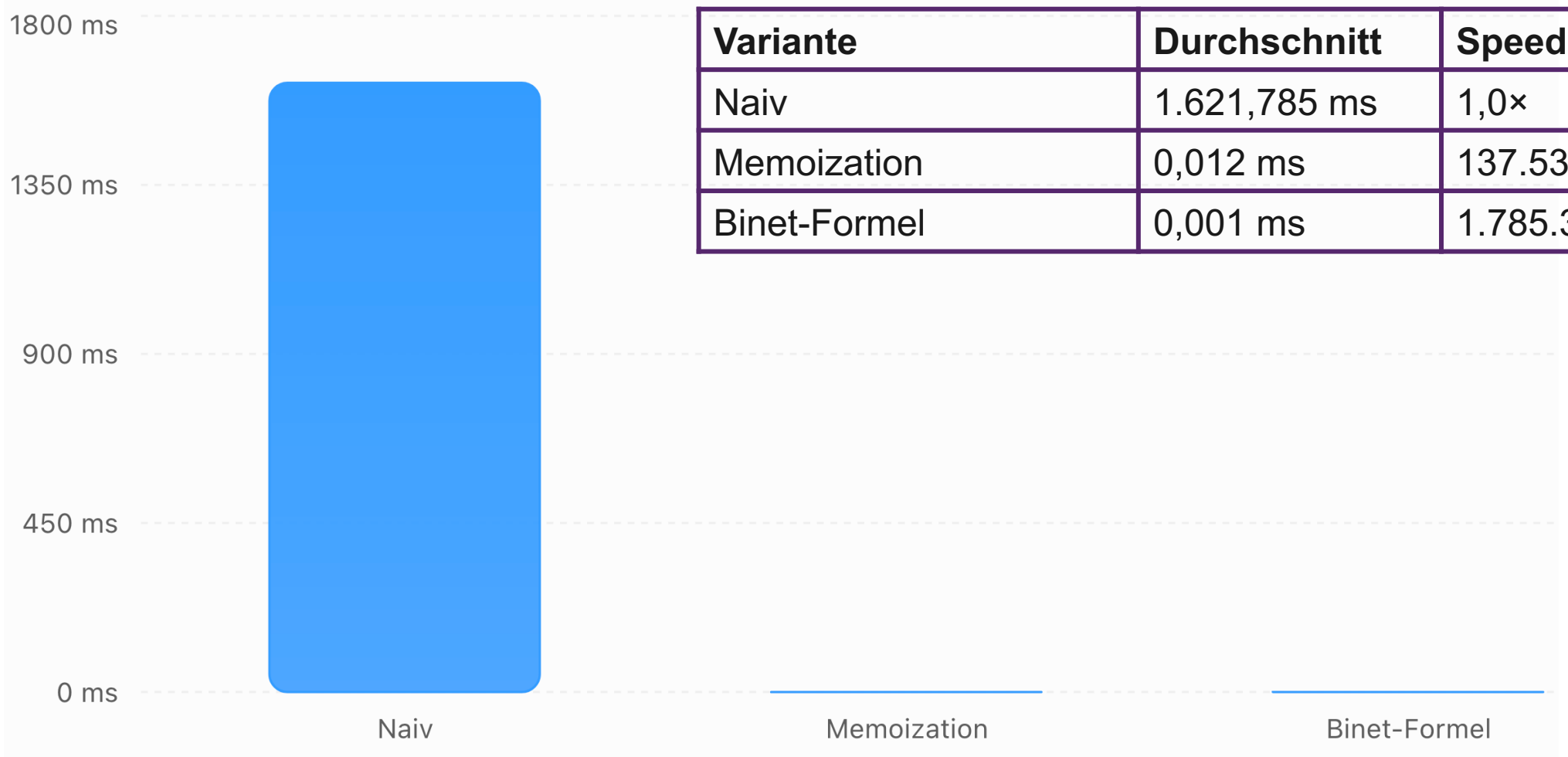
- **Binet's formula***

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}} \quad \text{where} \quad \varphi = \frac{1+\sqrt{5}}{2}, \qquad \psi = \frac{1-\sqrt{5}}{2}$$

```
static long fibFormula(int n) {
    double phi = (1 + Math.sqrt(5)) / 2;
    double psi = (1 - Math.sqrt(5)) / 2;

    return Math.round((Math.pow(phi, n) - Math.pow(psi, n)) / Math.sqrt(5));
}
```

Takeaway:
Suche einen alternativen Algorithmus bevor du die Implementierung optimierst.

Performance: Fibonacci: O(2ⁿ) → O(n) → O(1)



Variante	Durchschnitt	Speed-up vs naiv
Naiv	1.621,785 ms	1,0×
Memoization	0,012 ms	137.534,97×
Binet-Formel	0,001 ms	1.785.320,20×

🔥🔥🔥 DEMO: `Firework1_*/2_*/3_*/4_*/5_*` 🔥🔥🔥

- 🔥 1 String + → StringBuffer
- 🔥 2 Lineare → Binärsuche
- 🔥 3 Primzahlen → Sieb
- 🔥 4 Schleife → Formel
- 🔥 5 Rekursion → Memoization

🎯 Fazit

Performance Cheat Sheet

Falls es Performance-Probleme gibt, helfen diese Fragen:

1. Welche Methoden?
2. Auf welcher Datenstruktur?
3. Wie häufig?

Methoden	ArrayList	HashSet / HashMap
Membership `contains()`	O(n)	O(1)*
`add()` / `put()`	O(1)*	O(1)*
`removeFirst()`	O(n)	—
`get(i)`	O(1)	—

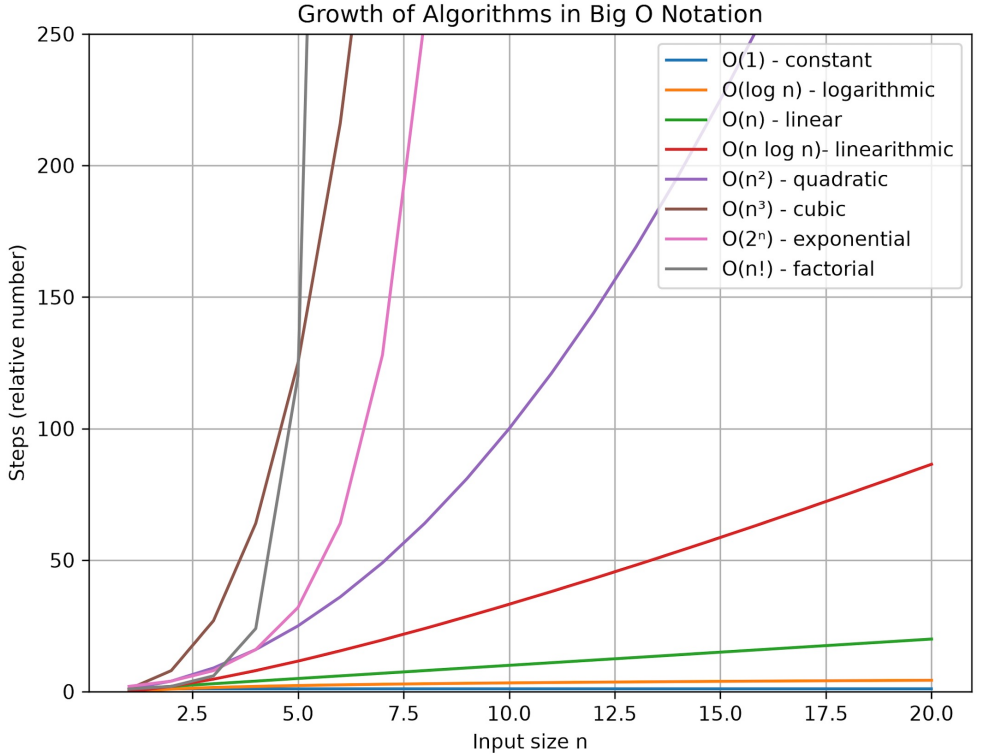
Performance Mental Model

1. **Wie ist die Komplexität meines Algorithmus?**

   `O(1)` < `O(log n)` < `O(n)` < `O(n log n)` < `O(n²)` < `O(2ⁿ)`

2. **Was nutze ich als Datenstruktur?**

   - `List` → Geordnete Sequenz
   - `Set` → Ideal für Membership
   - `Map` → Lookup per Schlüssel



3. **Was passiert, wenn der Datenbestand (massiv) wächst?**

Key Takeaway: Don't optimize what happens once. Optimize what happens very often.

Positives

- Optimieren kann (richtig viel) Spass machen
- Einordnung O-Notation wichtig, um Potenziale zu erkennen
- Erzielbare Gewinne sind enorm und machen einen grossen Unterschied – aber erst bei grösseren Datenbeständen.
- Es gibt viele clevere Techniken, die beim Optimieren helfen – es ist wertvoll, einige zu kennen:
  - Two Pointers
  - Look-Up-Maps / Caching
  - Sliding Windows und Präfixsummen
  - Ausgeklügelte Algorithmen und Datenstrukturen wie Tries
  - Mathematische Tricks

Bedenkenswertes

- Niemals ohne Plan agieren und nur für Dinge optimieren, wo Skalierungseffekte, also Datenbeständen >10.000 oder noch deutlich mehr eine Rolle spielen. Ausnahme: Redundante Einzelcalls.
- Manchmal kann es ziemlich nervenaufreibend sein, den richtigen Ansatz fehlerfrei umzusetzen.
- Allzu clevere Lösungen sind manchmal schwieriger nachvollzihbar
- Unbedingt ans Testing denken, damit Optimierung nicht nur den Code schneller machen, sondern dieser auch weiterhin korrekt funktioniert.

Fragen?

THANK YOU

Selbststudium

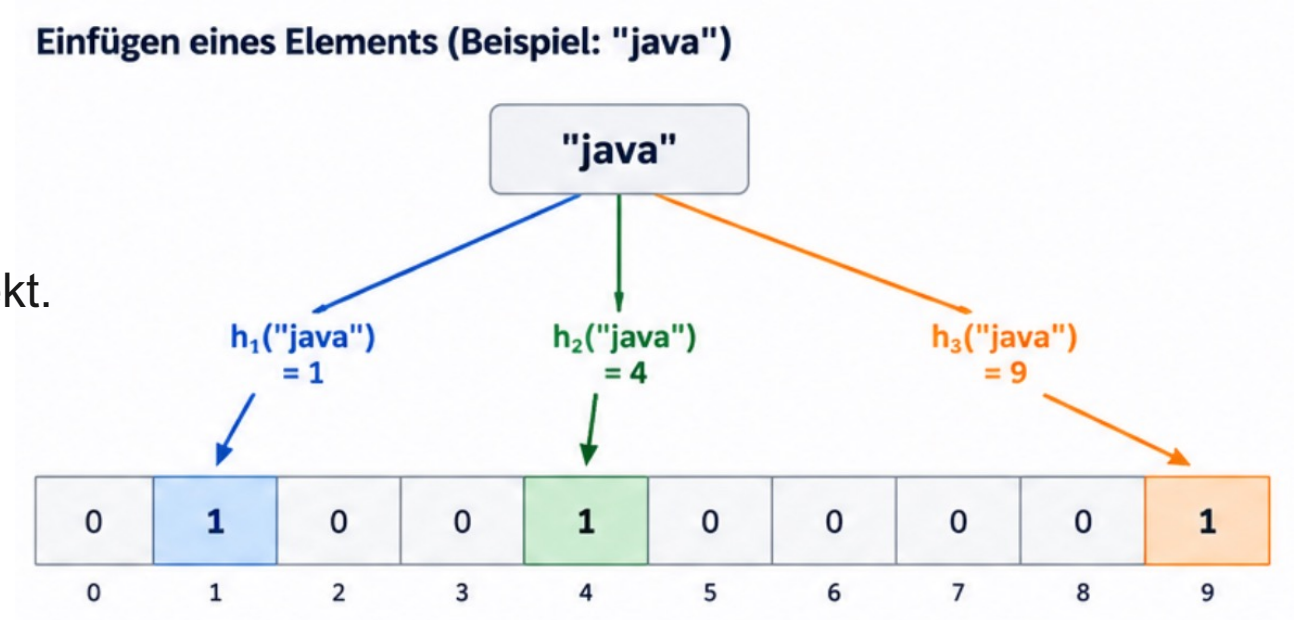
Think

Wann kann eine Näherung hilfreicher sein als eine exakte Antwort?

Bloom Filter

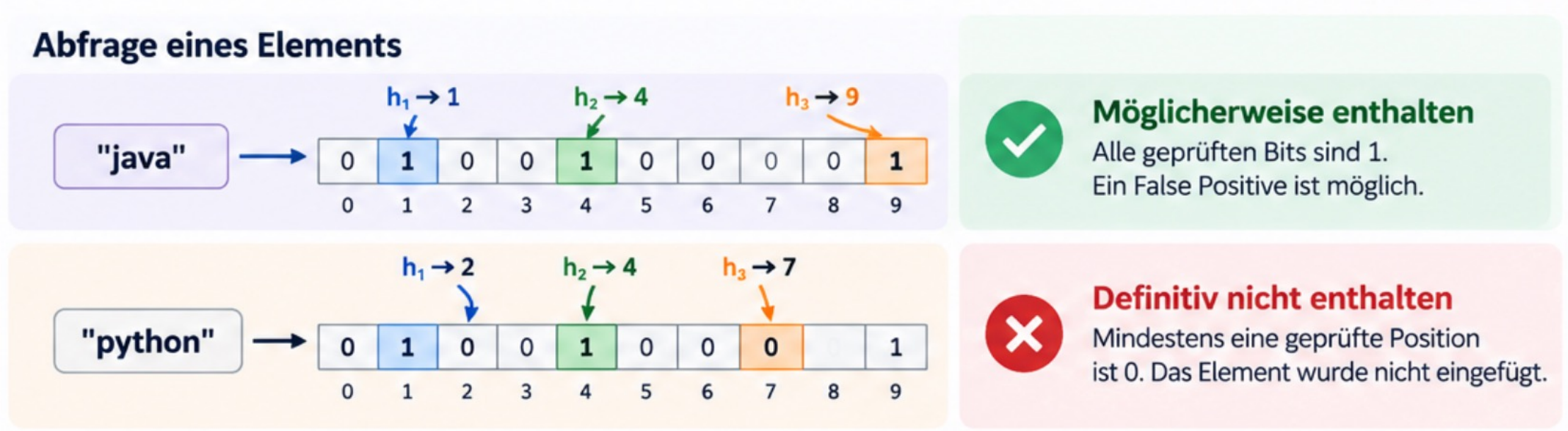
Bloom Filter – Wissen mit wenig Speicher

- **Das Problem**
  - Set / Map können schnell prüfen, ob Element enthalten.
  - Aber: Dafür müssen die Elemente selbst gespeichert werden.
  - Bei seher vielen Elementen ggf. Problematisch!
  - Was, wenn der verfügbare Speicher knapp ist?
- **Die Idee**
  - Ein Bloom Filter speichert die Elemente nicht direkt.
  - Mehrere Hashfunktionen bestimmen Positionen in einem Bitfeld.
  - Beim Einfügen werden diese Bits auf 1 gesetzt.



- **Kernidee**: Weniger Speicher durch eine bewusst ungenaue Darstellung der Menge.

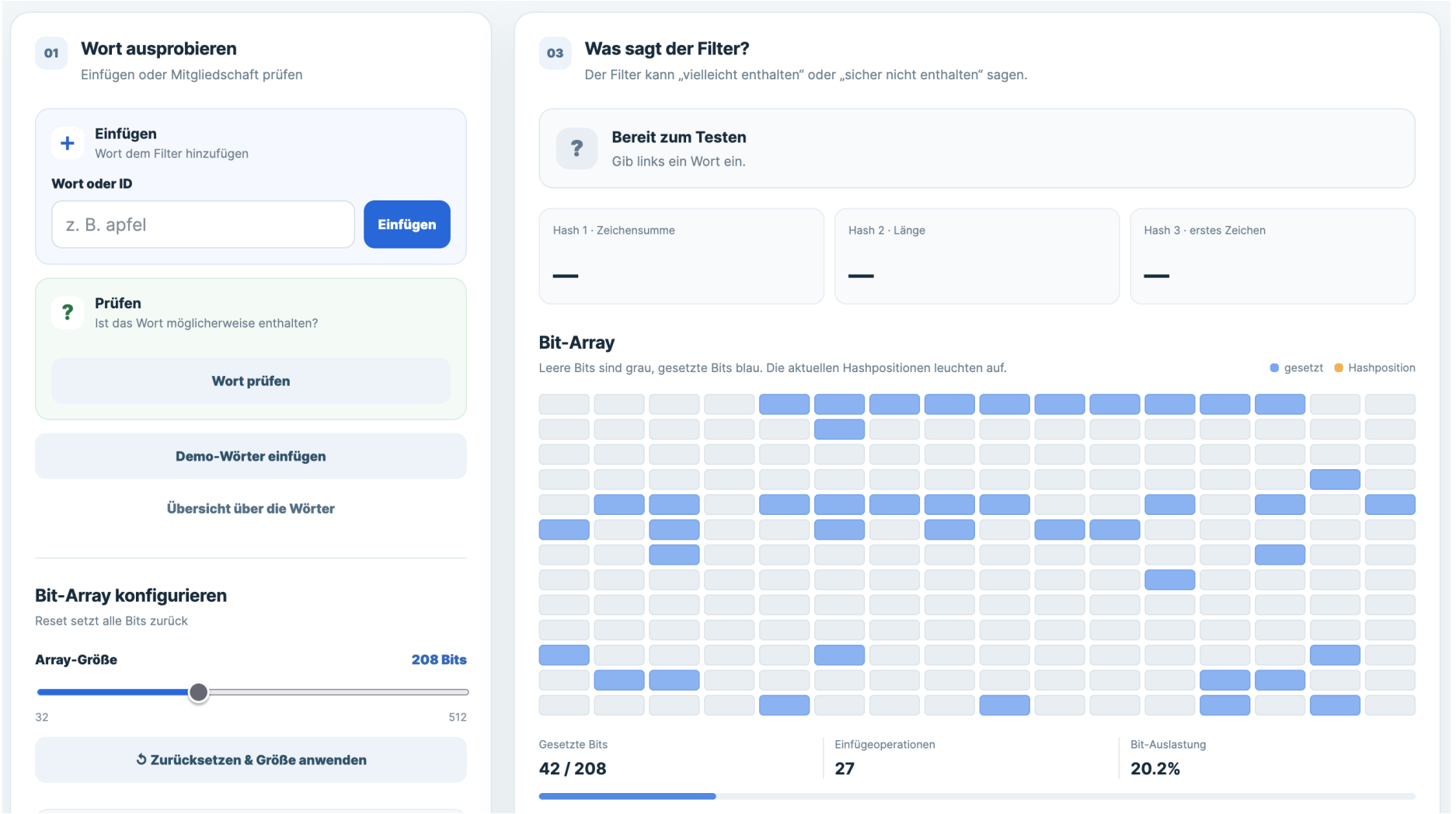
Bloom Filter – Schnell – aber nicht 100 % sichere Aussage



Der Trade-off

- Sehr geringer Speicherbedarf
- Schnelles Einfügen und Abfragen
- Mehr verfügbare Bits senken die Wahrscheinlichkeit von False Positives

DEMO: Bloom Filter Arena



JMH

Java Microbenchmark Harness

JMH – Performance messen statt vermuten

- **Einordnung**: Standard-Framework des OpenJDK für zuverlässige Microbenchmarks auf der JVM.
- **Motivation**: Kleine Performance-Unterschiede zwischen Implementierungen,, Datenstrukturen und Algorithmen messbar machen.
- **Mehr Aussagekraft**: Warm-up-Phasen, mehrere Messungen und statistische Auswertung berücksichtigen typische JVM-Effekte.
- **Praktischer Nutzen**: Optimierungen objektiv vergleichen und Performance-Regressionen früh erkennen.
- **Abgrenzung**: Misst gezielt kleine Codeabschnitte – ersetzt keine End-to-End- oder Lasttests.
- **Kernaussage**: Nicht raten, welcher Code schneller ist – messen, vergleichen und fundiert entscheiden.

„Performance-Optimierung braucht belastbare Messwerte – keine Bauchgefühle."

JMH – Performance messen statt vermuten

- **Motivation: Implementierungsdetails objektiv vergleichen**
  - Kleine Codevarianten hinsichtlich ihrer Performance vergleichen
  - Den Einfluss konkreter Implementierungsentscheidungen messbar machen
  - Herausfinden, welche Variante bei gleicher Funktionalität effizienter ist
  - Intuition durch reproduzierbare Messwerte ersetzen
  - Optimierungspotenziale identifizieren, bevor Änderungen ins Gesamtsystem übernommen werden
- **Beispiele für solche Vergleiche:**
  - `for`-Schleife vs. Stream
  - `String`-Konkatenation mit + vs. `StringBuilder`
  - Unterschiedliche Varianten einer Berechnung
  - Unterschiedliche API-Aufrufe für dieselbe Aufgabe

JMH – Java Microbenchmark Harness

Zuverlässige, reproduzierbare Performance-Messungen für Java

Was ist JMH?

- Offizielles Microbenchmark-Framework von OpenJDK
- Speziell für zuverlässige, reproduzierbare Messungen in der JVM entwickelt
- Abstrahiert typische Fehlerquellen bei Performance-Messungen
- Einfach zu verwenden, mächtig konfigurierbar

Warum überhaupt Microbenchmarks?

- Performance ist oft ein entscheidender Faktor (z. B. in Bibliotheken, Algorithmen, Hot Paths)
- Hilft, Design-Entscheidungen datenbasiert zu treffen
- Macht Verbesserungen messbar und sichtbar
- Verhindert „gefühlte“ statt echte Optimierungen
- Ideal, um kleine Unterschiede systematisch zu analysieren

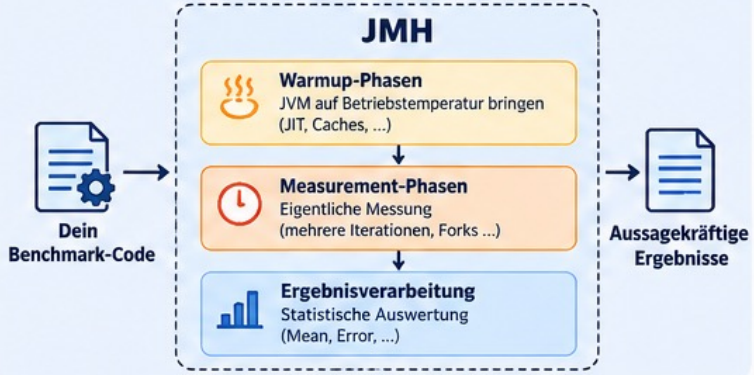
Deine Vorteile auf einen Blick

- Korrekte, aussagekräftige Messergebnisse
- Automatische Warmup- und Messphasen
- Vermeidet typische JVM-Fallen (z. B. Dead Code Elimination)
- Flexible Konfiguration (Threads, Forks, Modi ...)
- Klare, leicht interpretierbare Ergebnisse
- Unterstützt verschiedene Metriken

Typischer JMH-Workflow

1. **Benchmark schreiben**
   - @Benchmark Annotieren
   - State definieren
   - Parameter (optional)
2. **Kompilieren und ausführen**
   - via Maven / Gradle oder Kommandozeile
   - JMH übernimmt Warmup, Measurement, Forks, ...
3. **Ergebnisse auswerten**
   - Zeit, Durchsatz etc.
   - Statistische Kennzahlen
   - Vergleich verschiedener Ansätze

So funktioniert JMH (vereinfacht)



Typische Messmodi

- **AverageTime** – Durchschniftliche Zeit pro Operation
- **Throughput** – Anzahl Operationen pro Zeitenheit
- **SampleTime** – Verteilung der Einzelzeiten (Stichprobe)
- **SingleShotTime** – Einmalige Messung (z. B. Setup-Kosten)
- **All** – Alle Modi gleichzeitig

Minimales Beispiel

```
import org.openjdk.jmh.annotations.*;

@State(Scope.Thread)
public class MyBenchmark {
    private int value = 42;

    @Benchmark
    public int compute() {
        return value * 2;
    }
}
```

Wichtige Annotations (Auswahl)

Annotation	Bedeutung
@Benchmark	Markiert die zu messende Methode
@State	Definiert den Lebenszyklus und Scope des Zustands
@Param	Parameter für verschiedene Testvarianten
@Warmup	Konfiguration der Warmup-Phasen
@Measurement	Konfiguration der Mess-Phasen
@Fork	Anzahl der separaten JVM-Prozesse
@BenchmarkMode	Legt den Messmodus fest (z. B. Throughput)
@OutputTimeUnit	Zeiteinheit für die Ausgabe

Best Practices

- [x] Immer JMH statt selbstgebauter Timer verwenden
- [x] Warmup-Phasen nicht überspringen
- [x] Mehrere Forks nutzen (Standard: 1–5)
- [x] Realistische Daten und Szenarien verwenden
- [x] Ergebnisse statistisch betrachten (Mean, Error, ...)
- [x] Keine Seiteneffekte oder Dead Code erzeugen
- [x] Benchmarks isoliert und reproduzierbar ausführen

Messen. Verstehen. Besser machen.